TRABALHO DE CONCLUSÃO DE CURSO

Especialização em Ciência de Dados - Turma 2018.1 - Facens

Orientador:	**Fernando Vieira da Silva, M.Sc.**

Nome:	**José Diniz Junior**	**RA:**	**183134**
Nome:	Roberto Leonardo Rodrigues	**RA:**	060235

O objetivo deste trabalho é explorar e entender melhor as características de dados provindos de dois sensores (sensores fotovoltaicos e meteorológicos) a fim de se obter informações necessárias para a previsão de geração de energia para os próximos dez minutos.

1 - Análise dos dados

Para o presente trabalho será utilizado um conjunto de datasets, sendo eles:

Fotovoltaico.csv - referente a dados da estação fotovoltaica composta por quatro inversores solares, totalizando 8022697 registros.

Meteorologico.csv - referente a dados da estação meteorológica, totalizando 50502 registros.

Com dados coletados entre julho de 2018 e junho de 2019 pertencentes a Smart Campus Facens.

In [1]:

```
#CÉLULA TCC-01
#Importação de bibliotecas necessárias para execução do trabalho
import pandas as pd
import numpy as np
from datetime import datetime
from collections import Counter

#Gráficos
import matplotlib.pyplot as plt
from matplotlib.dates import DateFormatter, MinuteLocator, DayLocator, MonthLocator
import plotly.graph_objs as go
from plotly.offline import init_notebook_mode, iplot
init_notebook_mode(connected=True)
%matplotlib inline

from sklearn.preprocessing import MinMaxScaler
```

In [2]:

```
#CÉLULA TCC-02
#Método responsável por carregar os arquivos csv's
def loadDataFile(fileName, DateTimeIndex):
    #Convertendo a coluna de data para index e para o formato de DateTime
    if DateTimeIndex:
        return pd.read_csv(fileName, encoding='utf-8', index_col=0, parse_dates=[0])
    else:
        return pd.read_csv(fileName, encoding='utf-8')
```

In [3]:

```
#CÉLULA TCC-03
#Método responsável por salvar o arquivo em formato csv
def saveDataFile(dataFrame, fileName):
    dataFrame.to_csv(fileName, encoding='utf-8', sep=',', index=False)
```

In [4]:

```
#CÉLULA TCC-04
#Método responsável por plotar os dataFrames
def showDataPlot(dataFrame):
    plt.grid()
    plt.plot(dataFrame)
    plt.gca().legend(dataFrame.columns)
    plt.show()
```

1.1 - Análise do dataset fotovoltaico

In [5]:

```
#CÉLULA TCC-05
#Identificando todos os datasets referente ao período de quase 1 ano de dados fotovolta
icos
dataSetFotoVoltaico = ['07-2018.csv','08-2018.csv','09-2018.csv','10-2018.csv','11-201
8.csv','12-2018.csv'
                       ,'01-2019.csv','02-2019.csv','03-2019.csv','04-2019.csv','05-201
9.csv','06-2019.csv']
```

In [6]:

```
#CÉLULA TCC-06
#Realizando o carregamento dos dados fotovoltaicos
for idx, data in  enumerate(dataSetFotoVoltaico):
    path = 'input/fotovoltaico/{}'.format(data)
    if idx == 0:
        dfRawFoto = loadDataFile(path, True)
    else:
        dfRawFoto = pd.concat([dfRawFoto, loadDataFile(path, True)])
dfRawFoto.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 8022697 entries, 2018-07-01 00:00:01.590000 to 2019-06-01 0
0:00:05.503000
Data columns (total 8 columns):
painel           float64
energia_dia      float64
energia_ano      float64
energia_total    float64
potencia         float64
power_led        int64
solarnet_led     int64
solarweb_led     int64
dtypes: float64(5), int64(3)
memory usage: 550.9 MB
```

In [7]:

```
#CÉLULA TCC-07
#Selecionando campos
dfRawFoto.drop({
    #'painel',      #Campo representando cada inversor instalado, ou seja, 4 inversore
s, podendo ser 1,2,3 e 4;
    #'energia_dia', #Campo representando a energia acumulada por dia, retirado do própr
io inversor (Wh);
    'energia_ano',  #Campo representando a energia acumulada por ano (Wh);
    'energia_total',#Campo representando a energia acumulada desde o inicio da operação
do painel (Wh);
    #'potencia',    #Potência instantânea, é a potência da associação de módulos conect
ados ao inversor (W);
    'power_led',    #Led ...
    'solarnet_led', #Led ...
    'solarweb_led'  #Led ...
    },axis=1, inplace=True)
```

In [8]:

```
#CÉLULA TCC-08
#Verificando campos vazios
dfRawFoto.isna().sum()
```

Out[8]:

```
painel         503911
energia_dia    515245
potencia       515245
dtype: int64
```

In [9]:

```
#CÉLULA TCC-09
#Removendo campos vazios
dfRawFoto.dropna(inplace=True)
```

In [10]:

```
#CÉLULA TCC-10
#Selecionando apenas um inversor (painel)
dfRawFoto1 = dfRawFoto[dfRawFoto.painel == 1]
```

In [11]:

```
#CÉLULA TCC-11
#Ordenando o dataset pelo index, que representa o campo data
dfRawFoto1 = dfRawFoto1.sort_index()
dfRawFoto1.head()
```

Out[11]:

	painel	energia_dia	potencia
data			
2018-07-01 00:00:06.897	1.0	45990.0	0.0
2018-07-01 00:00:10.697	1.0	45990.0	0.0
2018-07-01 00:00:28.477	1.0	45990.0	0.0
2018-07-01 00:00:48.993	1.0	45990.0	0.0
2018-07-01 00:01:07.387	1.0	45990.0	0.0

In [12]:

```
#CÉLULA TCC-12
dfRawFoto1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1881412 entries, 2018-07-01 00:00:06.897000 to 2019-06-19 1
3:31:31.123000
Data columns (total 3 columns):
painel         float64
energia_dia    float64
potencia       float64
dtypes: float64(3)
memory usage: 57.4 MB
```

In [13]:

```
#CÉLULA TCC-13
#Criandno uma coluna nova para conter apenas a energia gerada em um determinado período
dfRawFoto1['energia'] = np.where(dfRawFoto1['potencia'] > 0, dfRawFoto1['energia_dia']
- dfRawFoto1['energia_dia'].shift(1), 0)
```

In [14]:

```
#CÉLULA TCC-14
#Removendo campos negativos
dfRawFoto1[dfRawFoto1 < 0] = 0
```

In [15]:

```
#CÉLULA TCC-15
#Removendo colunas desnecessárias
dfRawFoto1.drop({'energia_dia', 'potencia'},axis=1, inplace=True)
```

In [16]:

```
#CÉLULA TCC-16
#Verificando a quantidade de registros capturados por dia
qtdByDay = Counter(dfRawFoto1.index.date.astype('M8[D]'))
```

In [17]:

```
#CÉLULA TCC-17
#Aplicando um teto para descarte de dias com falhas em medições
court = .85  # abaixo do valor, será descartado o dia
lstDaysOut = []
for i in list(dict.fromkeys(qtdByDay.elements())):
    if qtdByDay[i] < 5760 * court : # 5760 - 15 em 15 segundo, todal de 5760 medições p
or dia
        lstDaysOut.append(i.astype(datetime))
print("Dias removidos")
lstDaysOut
```

```
Dias removidos
```

Out[17]:

```
[datetime.date(2018, 9, 4),
 datetime.date(2018, 9, 5),
 datetime.date(2018, 9, 23),
 datetime.date(2018, 10, 2),
 datetime.date(2018, 10, 4),
 datetime.date(2018, 10, 22),
 datetime.date(2018, 10, 23),
 datetime.date(2018, 12, 12),
 datetime.date(2018, 12, 13),
 datetime.date(2018, 12, 21),
 datetime.date(2018, 12, 22),
 datetime.date(2018, 12, 23),
 datetime.date(2018, 12, 24),
 datetime.date(2018, 12, 27),
 datetime.date(2019, 1, 30),
 datetime.date(2019, 1, 31),
 datetime.date(2019, 2, 5),
 datetime.date(2019, 2, 8),
 datetime.date(2019, 2, 22),
 datetime.date(2019, 2, 25),
 datetime.date(2019, 2, 28),
 datetime.date(2019, 3, 3),
 datetime.date(2019, 3, 5),
 datetime.date(2019, 3, 12),
 datetime.date(2019, 3, 13),
 datetime.date(2019, 3, 14),
 datetime.date(2019, 3, 28),
 datetime.date(2019, 4, 14),
 datetime.date(2019, 4, 15),
 datetime.date(2019, 6, 19)]
```

In [18]:

```
#CÉLULA TCC-18
#Removendo dias com medições abaixo do teto
dfRawFoto1['date'] = dfRawFoto1.index.date # coluna auxiliar
for i in lstDaysOut:# correndo todos os dias com falhas nas medições
    idx = dfRawFoto1[dfRawFoto1.date == i].index # obtendo o index
    dfRawFoto1.drop(idx, inplace=True) # removendo do dataset
dfRawFoto1.drop(['date'], axis=1, inplace=True) # removendo coluna auxiliar
```

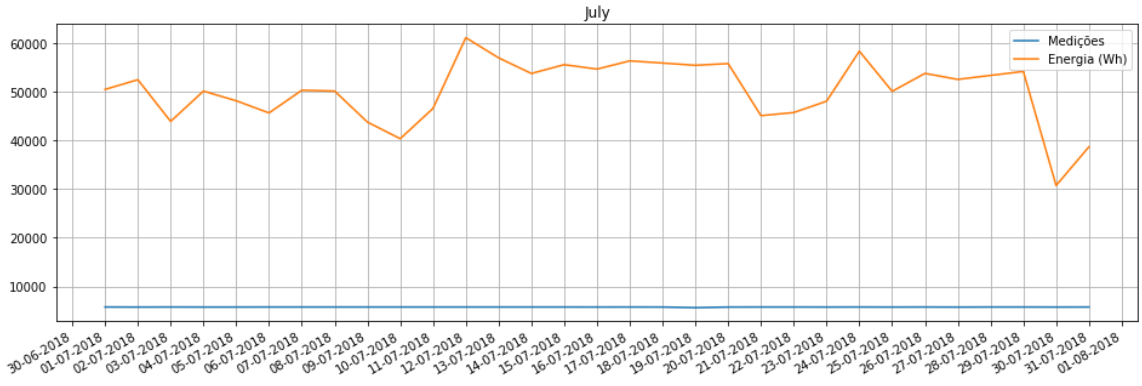
In [19]:

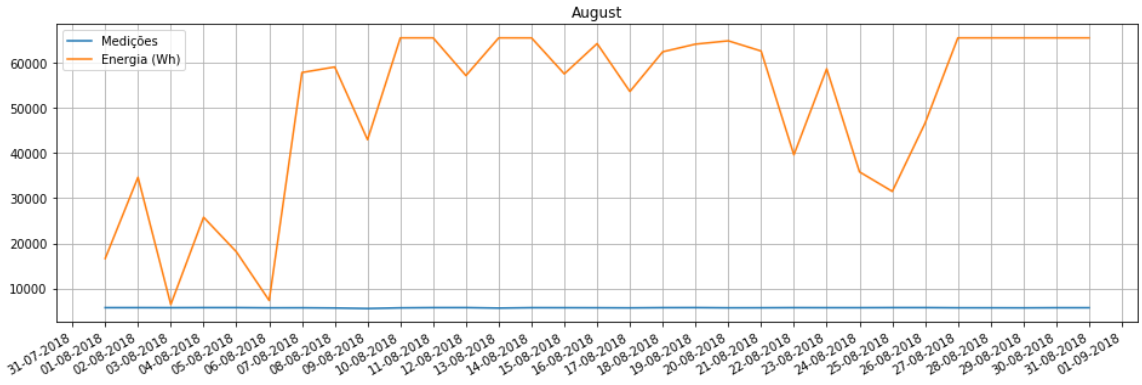
```
#CÉLULA TCC-19
dfRawFoto1.info()
```

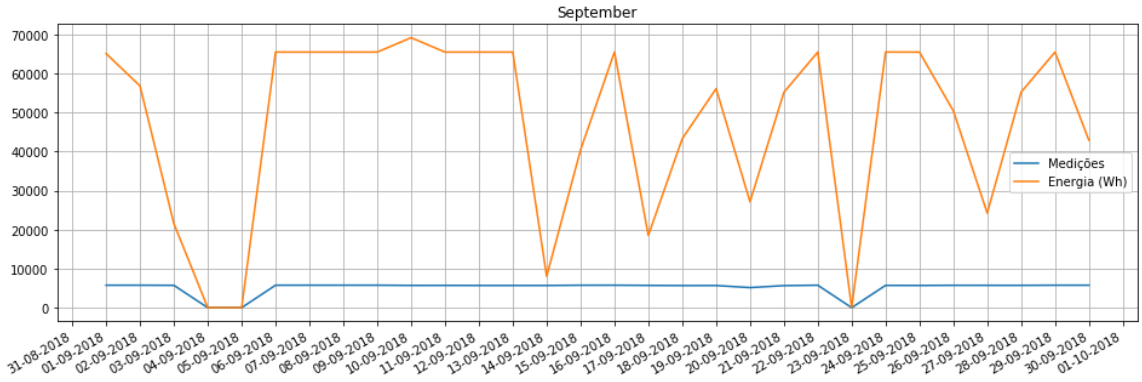
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1787516 entries, 2018-07-01 00:00:06.897000 to 2019-06-18 2
3:59:49.693000
Data columns (total 2 columns):
painel     float64
energia    float64
dtypes: float64(2)
memory usage: 40.9 MB
```

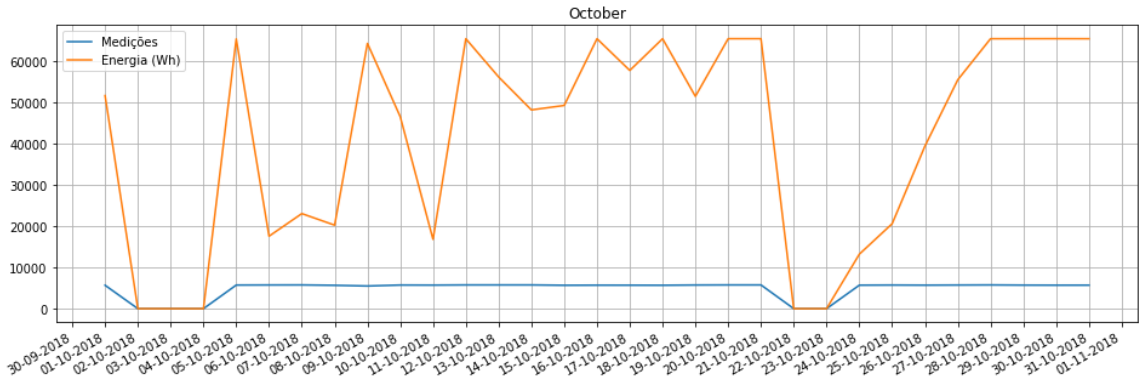
In [20]:

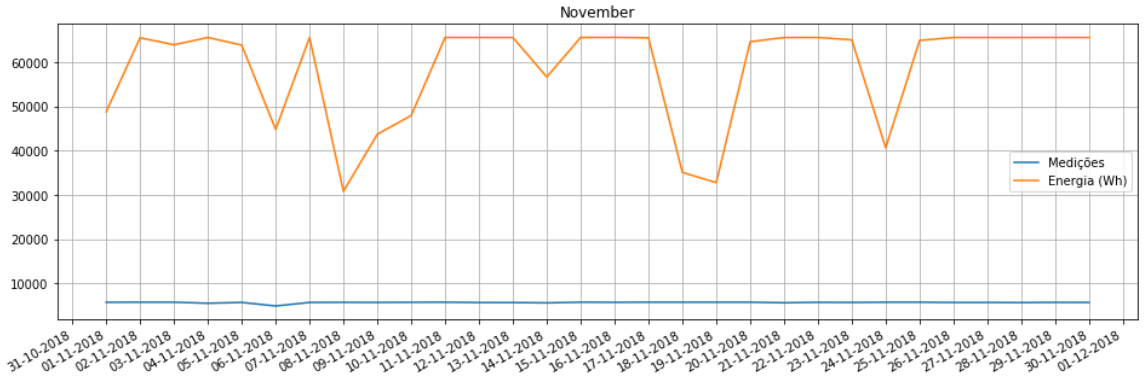
```
#CÉLULA TCC-20
#Gráfico mensal com a quantidade de energia acumulada por dia e a quantidade de medições registradas
for m in dfRawFoto1.groupby(by=[dfRawFoto1.index.month], sort=False):
    plt.figure(figsize=(16,5))
    plt.grid()
    #plt.plot(m[1].groupby(by=[m[1].index.date]).agg({"painel":'size',"energia":'sum'}))
    plt.plot(m[1].resample('D').agg({"painel":'size',"energia":'sum'}))#exibi datas excluidas
    plt.title(m[1].index.month_name()[0])
    plt.gca().legend(["Medições","Energia (Wh)"])
    plt.gca().xaxis.set_major_locator(DayLocator())
    plt.gca().xaxis.set_major_formatter(DateFormatter('%d-%m-%Y'))
    plt.gcf().autofmt_xdate()
    plt.show()
```

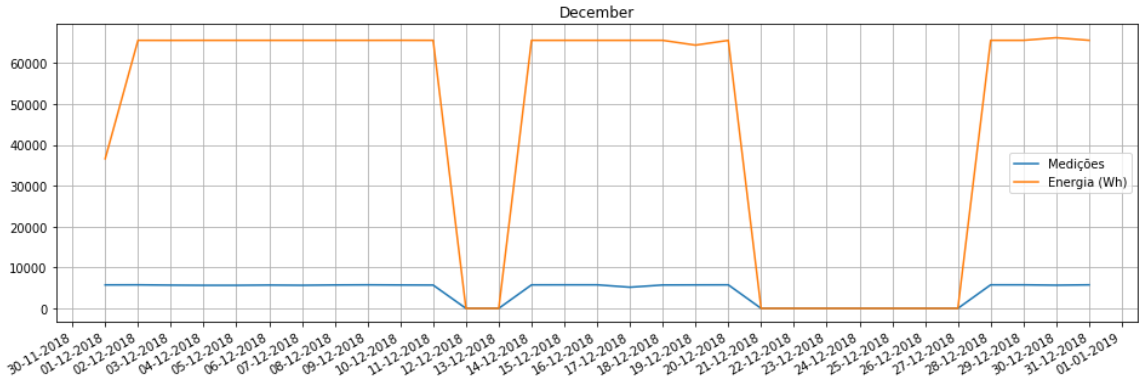

July
Medições
Energia (Wh)

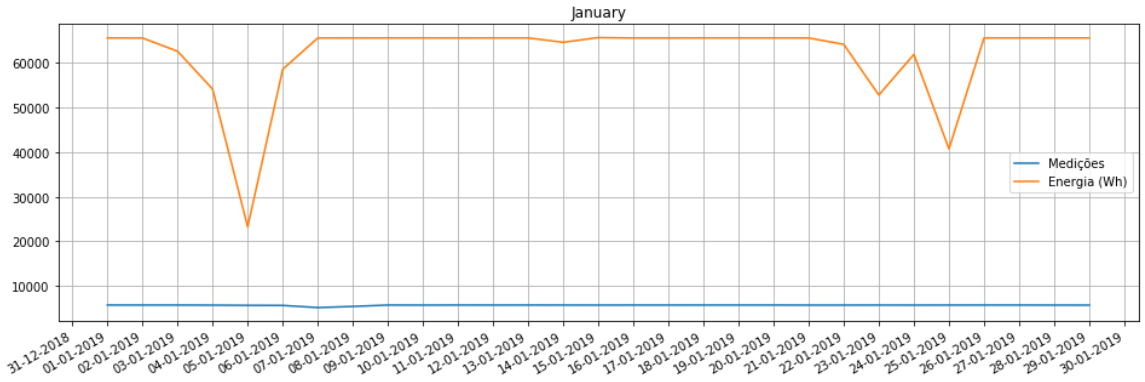

August
Medições
Energia (Wh)

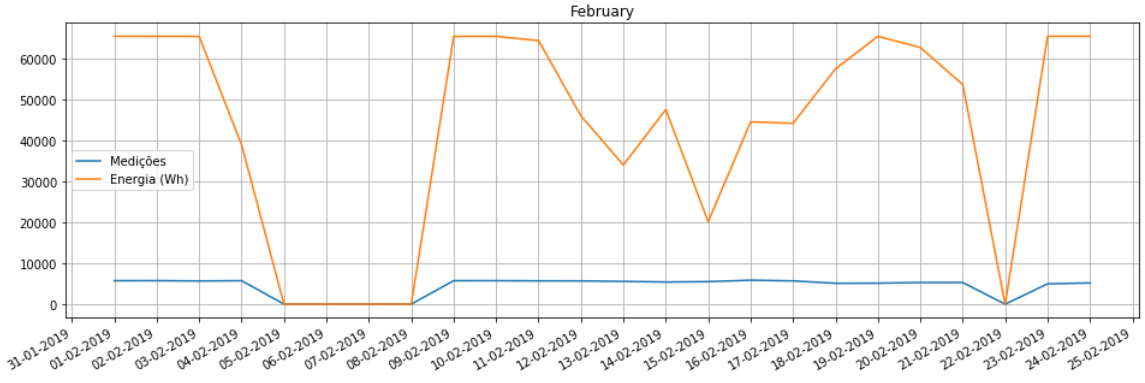

September
Medições
Energia (Wh)

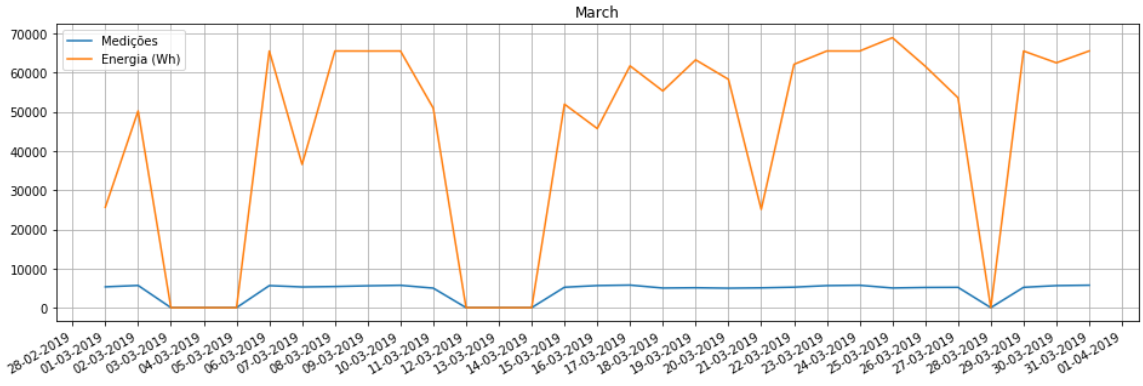

October
Medições
Energia (Wh)

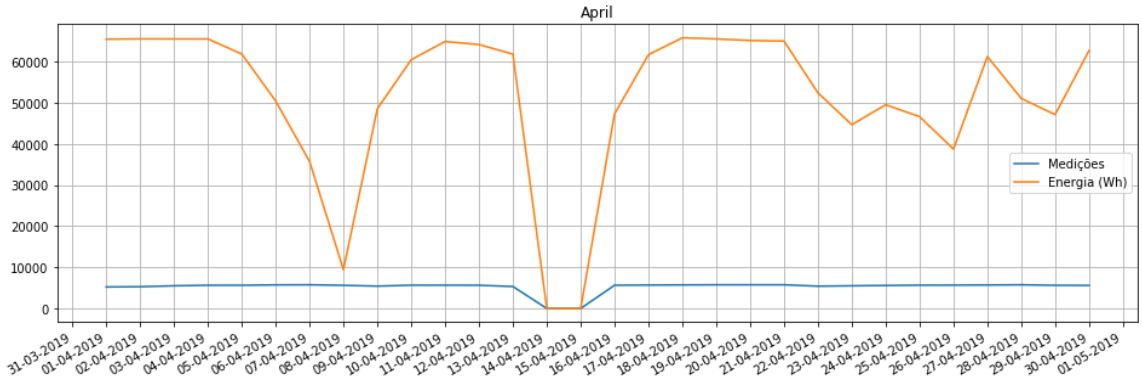

November
Medições
Energia (Wh)

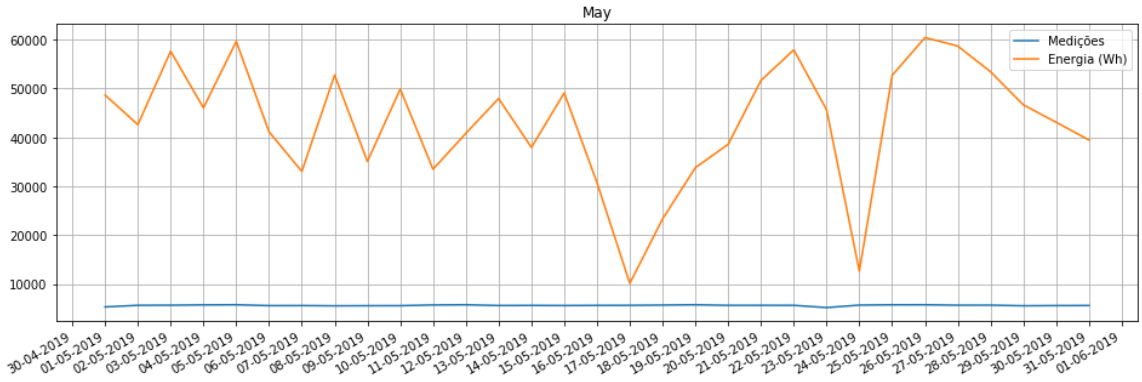

December
Medições
Energia (Wh)

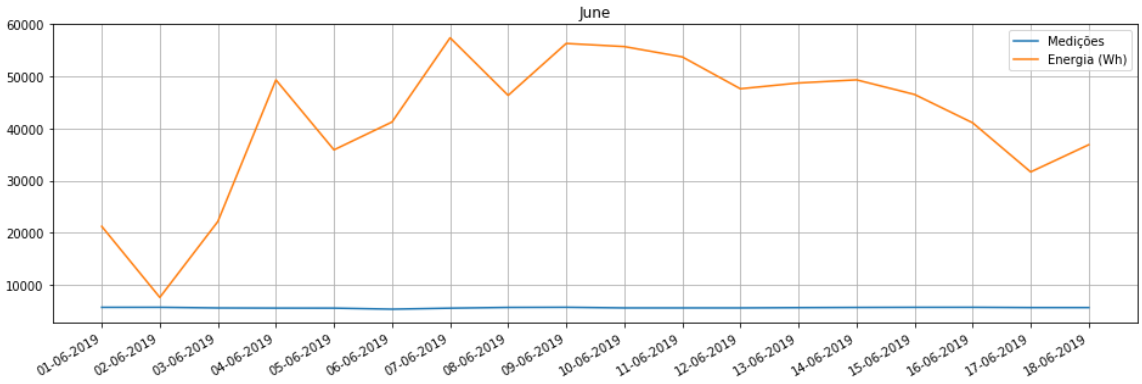

January
Medições
Energia (Wh)


February
Medições
Energia (Wh)


March
Medições
Energia (Wh)


April
Medições
Energia (Wh)


May
Medições
Energia (Wh)


June
Medições
Energia (Wh)

In [21]:

```
#CÉLULA TCC-21
#Realizando uma análise da energia acumulada em um dia
period = dfRawFoto1['2019-06-09':'2019-06-09']
print("Total de energia acumulada {:.0f} Wh".format(period.energia.sum()))

trace1 = period.groupby(by=[period.index.hour]).energia.sum().reset_index()
trace = go.Bar(x=trace1['data'],y=trace1['energia'])
data = [trace]
layout = go.Layout(
    title='Energia acumulada por hora',titlefont=dict(size=25),
    xaxis=dict(title='Hora',
               titlefont=dict(size=20),
               ticks='outside',
               tick0=0,
               dtick=1,
               ticklen=5,
               tickwidth=1,
              ),
    yaxis=dict(title='Energia (Wh)',titlefont=dict(size=20)))
iplot(go.Figure(data = [trace], layout=layout))
```

```
Total de energia acumulada 56280 Wh
```



1.2 - Análise do dataset meteorológico

In [22]:

```
#CÉLULA TCC-22
#Identificando todos os datasets referente ao período de quase 1 ano de dados meteoroló
gicos
dataSetMeteorologico = ['07-2018.csv','08-2018.csv','09-2018.csv','10-2018.csv','11-201
8.csv','12-2018.csv'
                        ,'01-2019.csv','02-2019.csv','03-2019.csv','04-2019.csv','05-201
9.csv','06-2019.csv']
```

In [23]:

```
#CÉLULA TCC-23
#Realizando o carregamento dos dados meteorológicos
for idx, data in  enumerate(dataSetMeteorologico):
    path = 'input/meteorologico/{}'.format(data)
    if idx == 0:
        dfRawMete = loadDataFile(path, True)
    else:
        dfRawMete = pd.concat([dfRawMete, loadDataFile(path, True)])
dfRawMete.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 50502 entries, 2018-07-01 00:00:07 to 2019-06-25 09:00:07
Data columns (total 14 columns):
deficit_vapor_vmin         50502 non-null float64
deficit_vapor_vmedio       50502 non-null float64
ponto_orvalho_vmin         50502 non-null float64
ponto_orvalho_vmedio       50502 non-null float64
precipitacao_valor         50502 non-null float64
radiacao_solar_valor       50497 non-null float64
temperatura_ar_vmin        50502 non-null float64
temperatura_ar_vmedio      50502 non-null float64
temperatura_ar_vmax        50502 non-null float64
umidade_ar_vmin            50502 non-null float64
umidade_ar_vmedio          50502 non-null float64
umidade_ar_vmax            50502 non-null float64
velocidade_vento_vmedio    50502 non-null float64
velocidade_vento_vmax      50502 non-null float64
dtypes: float64(14)
memory usage: 5.8 MB
```

In [24]:

```
#CÉLULA TCC-24
#Selecionando campos
dfRawMete.drop({
    'deficit_vapor_vmin',      #Campo representando a capacidade de absorção de água do
ar (valor mínimo);
    #'deficit_vapor_vmedio',   #Campo representando a capacidade de absorção de água do
ar (valor médio - ºC);
    'ponto_orvalho_vmin',      #Campo representando a temperatura em que fica saturado
 (valor mínimo - ºC);
    #'ponto_orvalho_vmedio',   #Campo representando a temperatura em que fica saturado
 (valor médio - ºC);
    #'precipitacao_valor',     #Campo representando a deposição de água para a superfíc
ie da Terra (mm);
    #'radiacao_solar_valor',   #Campo representando a energia radiante emitida pelo so
l;
    'temperatura_ar_vmin',     #Campo representando a temperatura (valor mínimo - ºC);
    #'temperatura_ar_vmedio',  #Campo representando a temperatura (valor médio - ºC);
    'temperatura_ar_vmax',     #Campo representando a temperatura (valor máximo - ºC);
    'umidade_ar_vmin',         #Campo representando a umidade do ar (valor mínimo - %);
    #'umidade_ar_vmedio',      #Campo representando a umidade do ar (valor médio - %);
    'umidade_ar_vmax',         #Campo representando a umidade do ar (valor máximo - %);
    #'velocidade_vento_vmedio',#Campo representando a velocidade do vento (valor médi
o);
    'velocidade_vento_vmax'    #Campo representando a velocidade do vento (valor máxim
o);
    },axis=1, inplace=True)
```

In [25]:

```
#CÉLULA TCC-25
#Verificando campos vazios
dfRawMete.isna().sum()
```

Out[25]:

```
deficit_vapor_vmedio       0
ponto_orvalho_vmedio       0
precipitacao_valor         0
radiacao_solar_valor       5
temperatura_ar_vmedio      0
umidade_ar_vmedio          0
velocidade_vento_vmedio    0
dtype: int64
```

In [26]:

```
#CÉLULA TCC-26
#Removendo campos vazios
dfRawMete.dropna(inplace=True)
```

In [27]:

```
#CÉLULA TCC-27
#Ordenando o data set pelo index, que representa o campo data
dfRawMete = dfRawMete.sort_index()
dfRawMete.head()
```

Out[27]:

	deficit_vapor_vmedio	ponto_orvalho_vmedio	precipitacao_valor	radiacao_
data				
2018-07-01 00:00:07	0.33	11.7	0.0	0.0
2018-07-01 00:10:07	0.31	12.0	0.0	0.0
2018-07-01 00:20:07	0.32	12.0	0.0	0.0
2018-07-01 00:30:06	0.31	11.9	0.0	0.0
2018-07-01 00:40:07	0.29	11.9	0.0	0.0

In [28]:

```
#CÉLULA TCC-28
#Verificando a quantidade de registros capturados por dia
qtdByDayMete = Counter(dfRawMete.index.date.astype('M8[D]'))
```

In [29]:

```
#CÉLULA TCC-29
#Aplicando um teto para descarte de dias com falhas em medições
lstDaysOutMete = []
for i in list(dict.fromkeys(qtdByDayMete.elements())):
    if qtdByDayMete[i] < 144 * court : # 144 - 10 em 10 minutos, todal de 144 medições
 por dia
        lstDaysOutMete.append(i.astype(datetime))
print("Dias removidos")
lstDaysOutMete
```

```
Dias removidos
```

Out[29]:

```
[datetime.date(2018, 8, 19),
 datetime.date(2018, 10, 22),
 datetime.date(2018, 10, 23),
 datetime.date(2018, 12, 1),
 datetime.date(2018, 12, 2),
 datetime.date(2018, 12, 3),
 datetime.date(2018, 12, 4),
 datetime.date(2018, 12, 5),
 datetime.date(2018, 12, 6),
 datetime.date(2018, 12, 7),
 datetime.date(2018, 12, 8),
 datetime.date(2018, 12, 10),
 datetime.date(2018, 12, 11),
 datetime.date(2018, 12, 13),
 datetime.date(2018, 12, 14),
 datetime.date(2018, 12, 15),
 datetime.date(2018, 12, 16),
 datetime.date(2018, 12, 17),
 datetime.date(2019, 2, 11),
 datetime.date(2019, 6, 25)]
```

In [30]:

```
#CÉLULA TCC-30
#Removendo dias com medições abaixo do teto
dfRawMete['date'] = dfRawMete.index.date # coluna auxiliar
for i in lstDaysOutMete:# correndo todos os dias com falhas nas medições
    idx = dfRawMete[dfRawMete.date == i].index # obtendo o index
    dfRawMete.drop(idx, inplace=True) # removendo do dataset
dfRawMete.drop(['date'], axis=1, inplace=True) # removendo coluna auxiliar
```

In [31]:

```
#CÉLULA TCC-31
dfRawMete.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 48505 entries, 2018-07-01 00:00:07 to 2019-06-24 23:50:06
Data columns (total 7 columns):
deficit_vapor_vmedio       48505 non-null float64
ponto_orvalho_vmedio       48505 non-null float64
precipitacao_valor         48505 non-null float64
radiacao_solar_valor       48505 non-null float64
temperatura_ar_vmedio      48505 non-null float64
umidade_ar_vmedio          48505 non-null float64
velocidade_vento_vmedio    48505 non-null float64
dtypes: float64(7)
memory usage: 3.0 MB
```

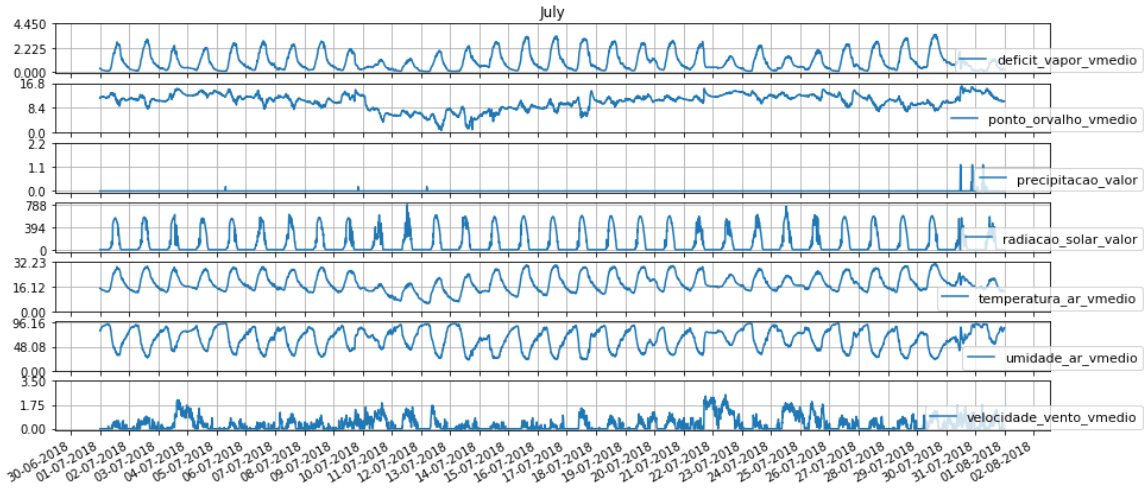
In [32]:

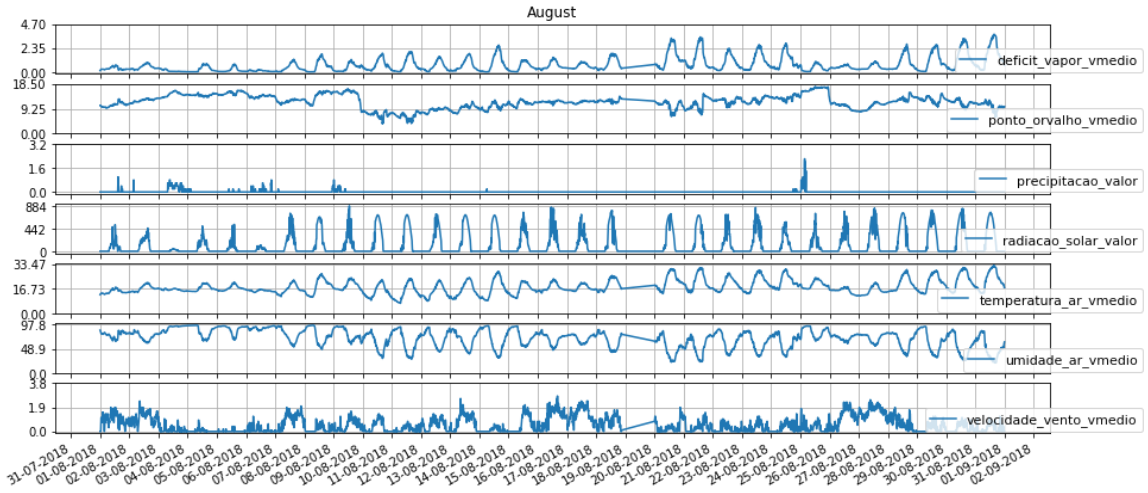
```
#CÉLULA TCC-32
#Método responsável por plotar gráficos distintos por colunas
def showDataPlotGroup(dataFrame):
    values = dataFrame.values
    i = 1
    plt.figure(figsize=(15,7))
    for column in dataFrame.columns:
        plt.subplot(len(dataFrame.columns), 1, i)
        if i == 1:
            plt.title(dataFrame.index.month_name()[0])
        plt.plot(dataFrame[column])
        plt.legend((column,), loc='lower right', bbox_to_anchor=(1.1, -0.1), prop={'siz
e': 11})
        plt.gca().xaxis.set_major_locator(DayLocator())#DayLocator(interval=5)
        plt.gca().xaxis.set_major_formatter(DateFormatter('%d-%m-%Y'))
        plt.gcf().autofmt_xdate()
        plt.yticks(np.arange(0, dataFrame[column].max()+2, step=(dataFrame[column].max
()+1)/2))
        i += 1
        plt.grid()

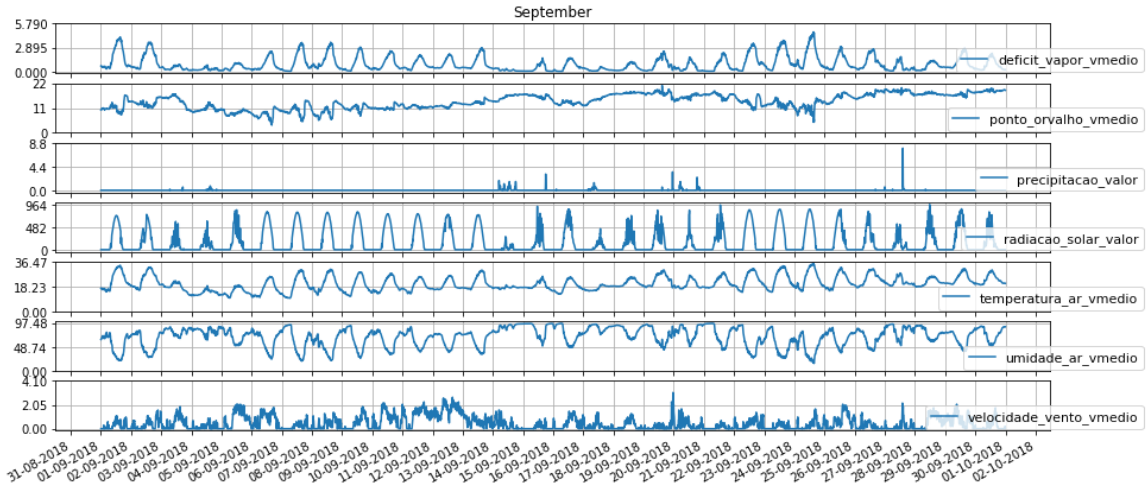
    plt.show()
```

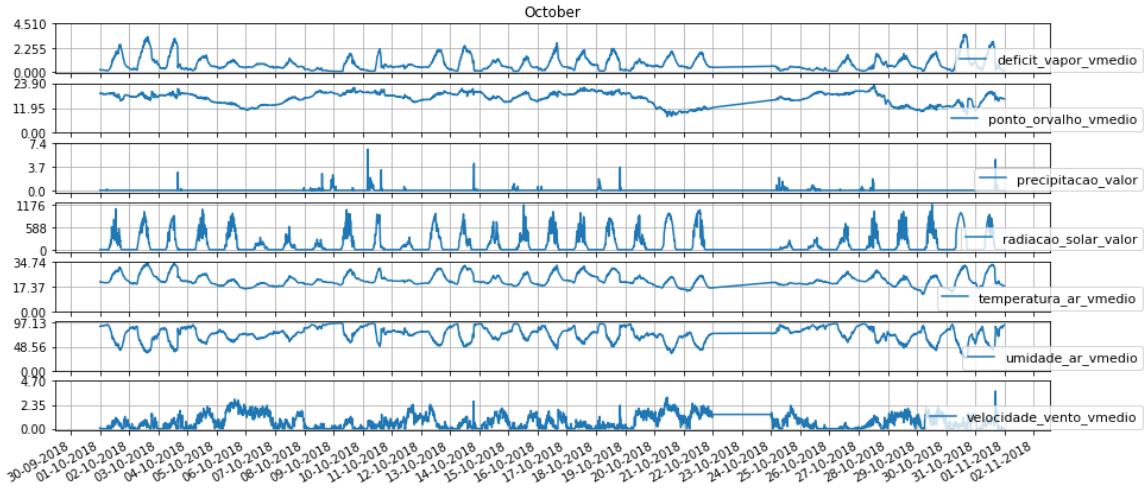
In [33]:

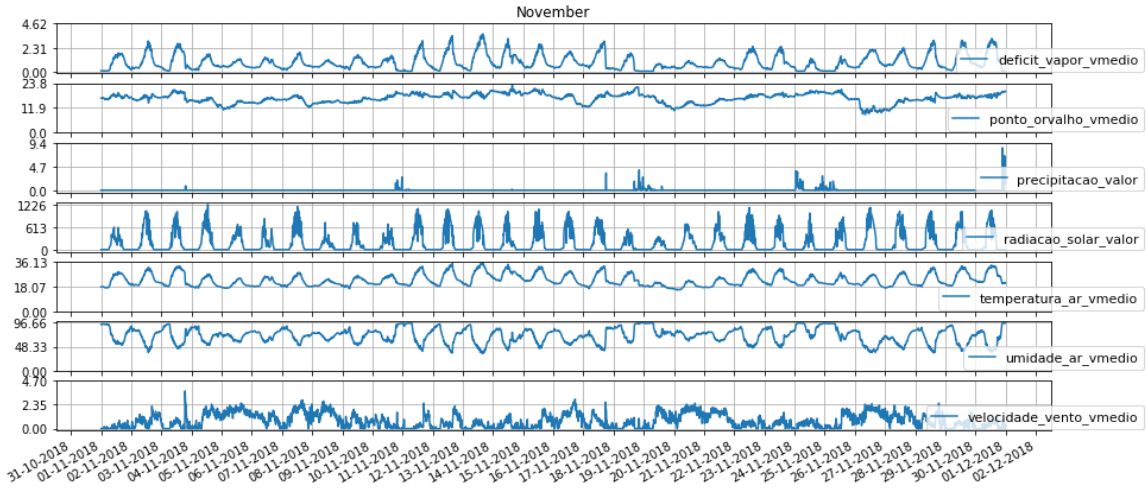
```
#CÉLULA TCC-33
for m in dfRawMete.groupby(by=[dfRawMete.index.month], sort=False):
    showDataPlotGroup(m[1][[
        #'deficit_vapor_vmin',
        'deficit_vapor_vmedio',
        #'ponto_orvalho_vmin',
        'ponto_orvalho_vmedio',
        'precipitacao_valor',
        'radiacao_solar_valor',
        #'temperatura_ar_vmin',
        'temperatura_ar_vmedio',
        #'temperatura_ar_vmax',
        #'umidade_ar_vmin',
        'umidade_ar_vmedio',
        #'umidade_ar_vmax',
        'velocidade_vento_vmedio',
        #'velocidade_vento_vmax',
        ]]
    )
```

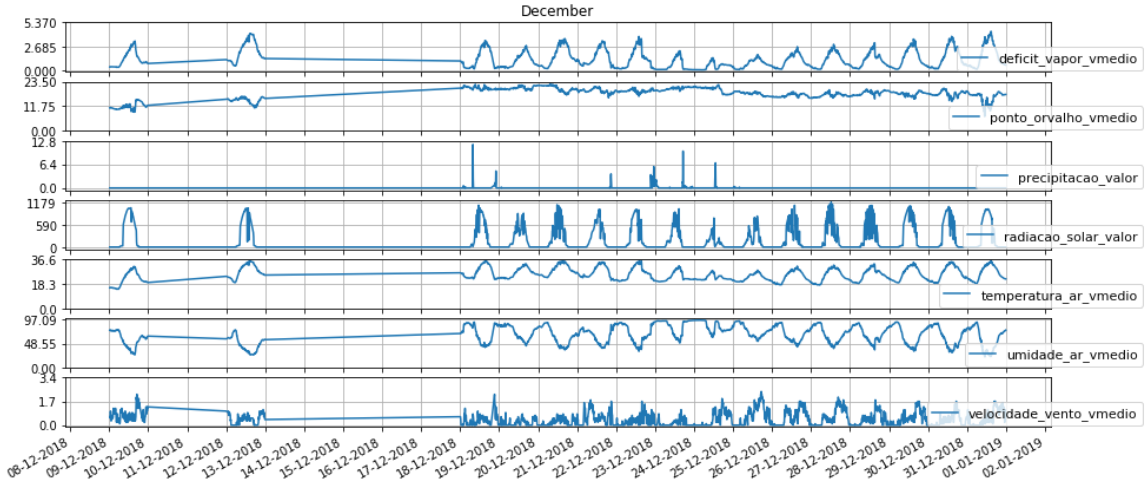

July
deficit_vapor_vmedio
ponto_orvalho_vmedio
precipitacao_valor
radiacao_solar_valor
temperatura_ar_vmedio
umidade_ar_vmedio
velocidade_vento_vmedio

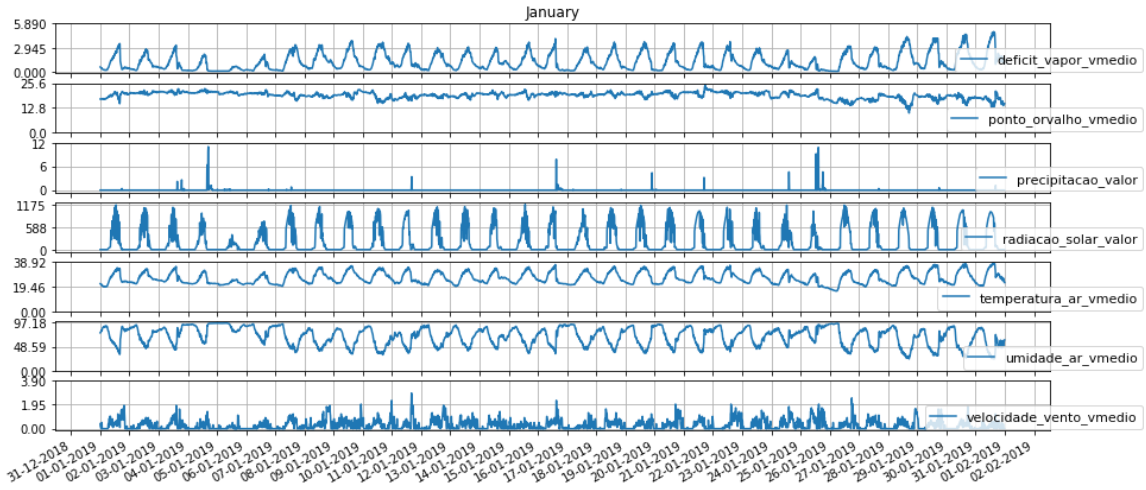

August
deficit_vapor_vmedio
ponto_orvalho_vmedio
precipitacao_valor
radiacao_solar_valor
temperatura_ar_vmedio
umidade_ar_vmedio
velocidade_vento_vmedio

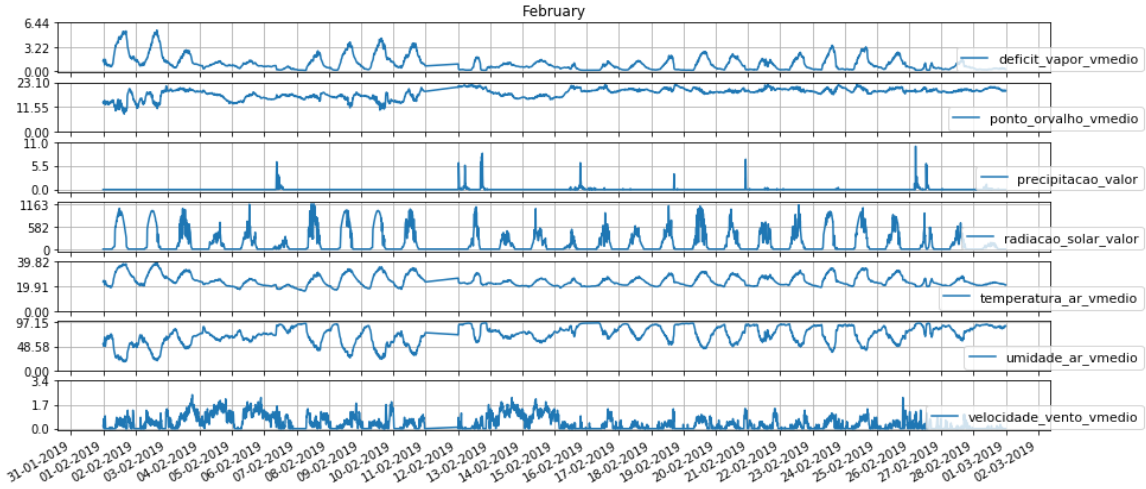

September
deficit_vapor_vmedio
ponto_orvalho_vmedio
precipitacao_valor
radiacao_solar_valor
temperatura_ar_vmedio
umidade_ar_vmedio
velocidade_vento_vmedio

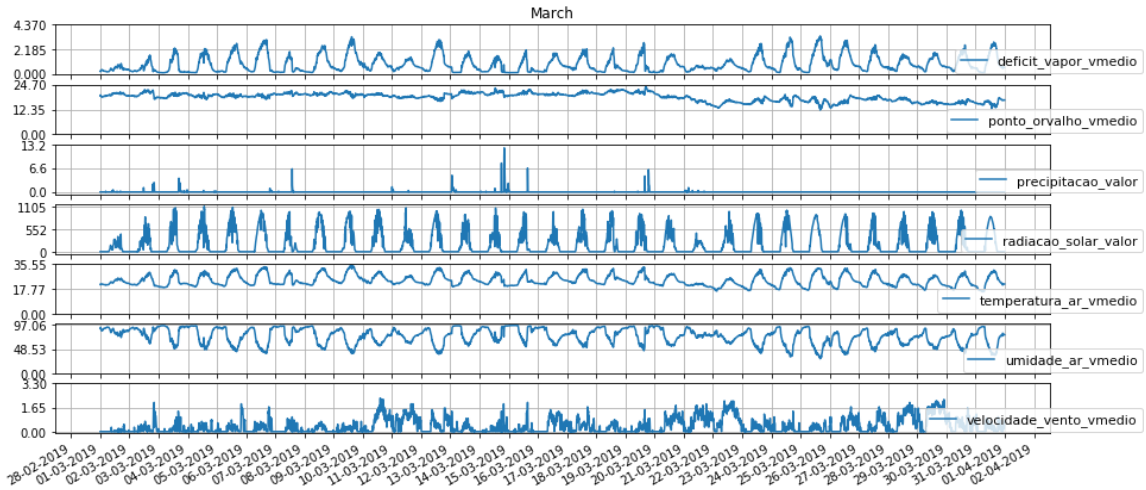

October
deficit_vapor_vmedio
ponto_orvalho_vmedio
precipitacao_valor
radiacao_solar_valor
temperatura_ar_vmedio
umidade_ar_vmedio
velocidade_vento_vmedio

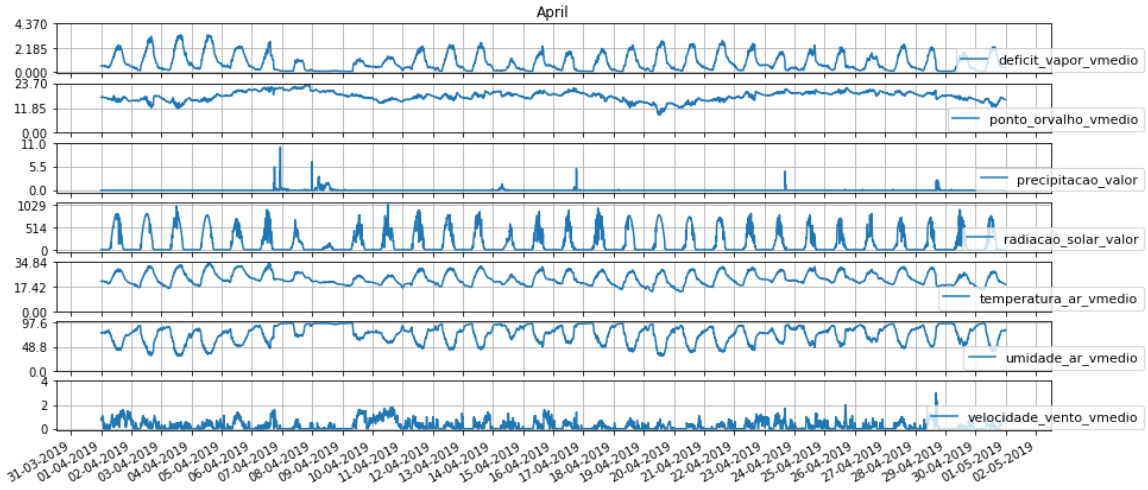

November
deficit_vapor_vmedio
ponto_orvalho_vmedio
precipitacao_valor
radiacao_solar_valor
temperatura_ar_vmedio
umidade_ar_vmedio
velocidade_vento_vmedio

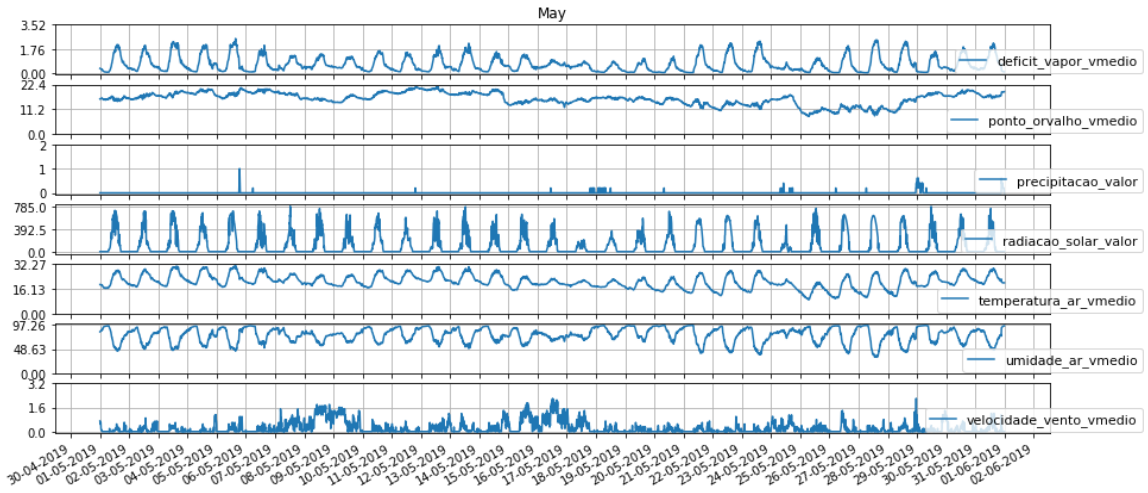

December
deficit_vapor_vmedio
ponto_orvalho_vmedio
precipitacao_valor
radiacao_solar_valor
temperatura_ar_vmedio
umidade_ar_vmedio
velocidade_vento_vmedio

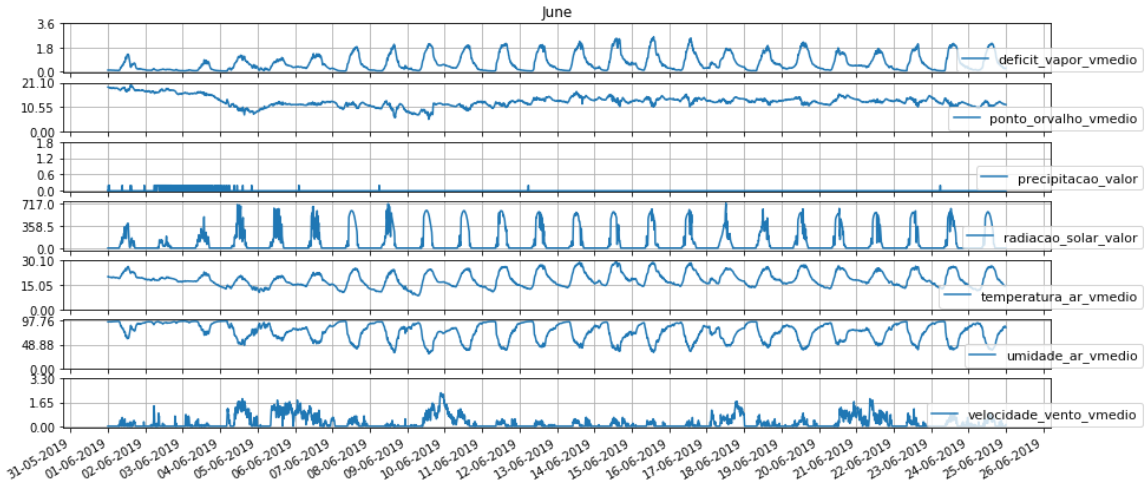

January
deficit_vapor_vmedio
ponto_orvalho_vmedio
precipitacao_valor
radiacao_solar_valor
temperatura_ar_vmedio
umidade_ar_vmedio
velocidade_vento_vmedio


February
deficit_vapor_vmedio
ponto_orvalho_vmedio
precipitacao_valor
radiacao_solar_valor
temperatura_ar_vmedio
umidade_ar_vmedio
velocidade_vento_vmedio


March
deficit_vapor_vmedio
ponto_orvalho_vmedio
precipitacao_valor
radiacao_solar_valor
temperatura_ar_vmedio
umidade_ar_vmedio
velocidade_vento_vmedio







1.3 - Unificando os datasets

Como os datasets apresentam medições distintas em tempos e frequência diferentes, como demonstra a representação abaixo:

- Fotovoltaico - apresenta em média 1 medição a cada 15 a 20 segundos, totalizando em média 240 medições por hora;
- Meteorologico - apresenta em média 1 mediação a cada 10 minutos, totalizando em média 6 medições por hora;

Será utilizado a tecnica de UpSample para os dados fotovoltaicos afim de centralizar as medições em um tempo maior, e para os dados meteorológicos a tecnica de DownSample para ajustar as medições em períodos faltantes, visando também um mesmo padrão de tempo nos dois datasets.

1.3.1 - DownSample

In [34]:

```
#CÉLULA TCC-34
dfRawFoto1.head()
```

Out[34]:

	painel	energia
data		
2018-07-01 00:00:06.897	1.0	0.0
2018-07-01 00:00:10.697	1.0	0.0
2018-07-01 00:00:28.477	1.0	0.0
2018-07-01 00:00:48.993	1.0	0.0
2018-07-01 00:01:07.387	1.0	0.0

In [35]:

```
#CÉLULA TCC-35
#Realizando ajuste das medicções para uma frequencia padrão de 10 min
freq = '10T' #1D - 1 dia,  10T - 10 min , 60T - 1 h   ou ('60min', on='TIME')

downFoto = dfRawFoto1.resample(freq) #de 10 em 10 minutos cravados
newDownFoto = downFoto.agg({
    "painel":'max',
    "energia":'sum' # realizando a somatória dos valores medidos, para se obter o valor
gerado em 10 min
})
interDownFoto = newDownFoto.interpolate(method='linear')
interDownFoto.head()
```

Out[35]:

	painel	energia
data		
2018-07-01 00:00:00	1.0	0.0
2018-07-01 00:10:00	1.0	0.0
2018-07-01 00:20:00	1.0	0.0
2018-07-01 00:30:00	1.0	0.0
2018-07-01 00:40:00	1.0	0.0

In [36]:

```
#CÉLULA TCC-36
interDownFoto.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 50832 entries, 2018-07-01 00:00:00 to 2019-06-18 23:50:00
Freq: 10T
Data columns (total 2 columns):
painel     50832 non-null float64
energia    50832 non-null float64
dtypes: float64(2)
memory usage: 3.7 MB
```

In [37]:

```
#CÉLULA TCC-37
#Removendo novamente, pois o resample gera novamente os dias, dias com medições abaixo
 do teto
interDownFoto['date'] = interDownFoto.index.date # coluna auxiliar
for i in lstDaysOut:# correndo todos os dias com falhas nas medições
    idx = interDownFoto[interDownFoto.date == i].index # obtendo o index
    interDownFoto.drop(idx, inplace=True) # removendo do dataset
interDownFoto.drop(['date'], axis=1, inplace=True) # removendo coluna auxiliar
```

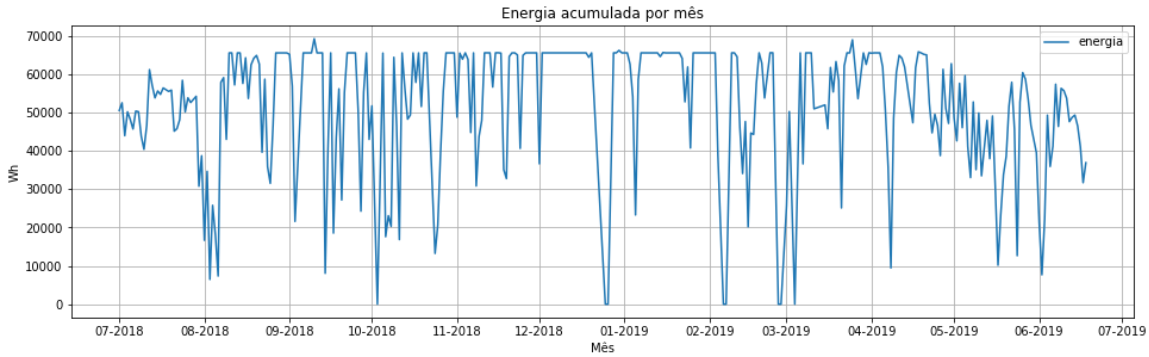
In [38]:

```
#CÉLULA TCC-38
interDownFoto.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 46656 entries, 2018-07-01 00:00:00 to 2019-06-18 23:50:00
Data columns (total 2 columns):
painel     46656 non-null float64
energia    46656 non-null float64
dtypes: float64(2)
memory usage: 1.1 MB
```

In [39]:

```
#CÉLULA TCC-39
#Gráfico representanto toda a energia acumulada nos meses capturados
plt.figure(figsize=(16,5))
plt.title("Energia acumulada por mês")
showDat = interDownFoto.groupby(by=interDownFoto.index.date).agg({"energia":'sum'})
plt.gca().set_xlabel('Mês')
plt.gca().set_ylabel('Wh')
plt.gca().xaxis.set_major_locator(MonthLocator())
plt.gca().xaxis.set_major_formatter(DateFormatter('%m-%Y'))
plt.gcf().autofmt_xdate()
showDataPlot(showDat)
```



1.3.2 - UpSample

In [40]:

```
#CÉLULA TCC-40
#Realizando ajuste das medicções para uma frequencia padrão de 10 min
downMete = dfRawMete.resample(freq) #de 10 em 10 minutos cravados
newDownMete = downMete.agg({
    #'deficit_vapor_vmin':'min',
    'deficit_vapor_vmedio':'mean',
    #'ponto_orvalho_vmin':'min',
    'ponto_orvalho_vmedio':'mean',
    'precipitacao_valor':'mean',
    'radiacao_solar_valor':'mean',
    #'temperatura_ar_vmin':'min',
    'temperatura_ar_vmedio':'mean',
    #'temperatura_ar_vmax':'max',
    #'umidade_ar_vmin':'min',
    'umidade_ar_vmedio':'mean',
    #'umidade_ar_vmax':'max',
    'velocidade_vento_vmedio':'mean',
    #'velocidade_vento_vmax':'max'
})
interDownMete = newDownMete.interpolate(method='linear')
interDownMete.head()
```

Out[40]:

	deficit_vapor_vmedio	ponto_orvalho_vmedio	precipitacao_valor	radiacao_
data				
2018-07-01 00:00:00	0.33	11.7	0.0	0.0
2018-07-01 00:10:00	0.31	12.0	0.0	0.0
2018-07-01 00:20:00	0.32	12.0	0.0	0.0
2018-07-01 00:30:00	0.31	11.9	0.0	0.0
2018-07-01 00:40:00	0.29	11.9	0.0	0.0

In [41]:

```
#CÉLULA TCC-41
interDownMete.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 51696 entries, 2018-07-01 00:00:00 to 2019-06-24 23:50:00
Freq: 10T
Data columns (total 7 columns):
deficit_vapor_vmedio       51696 non-null float64
ponto_orvalho_vmedio       51696 non-null float64
precipitacao_valor         51696 non-null float64
radiacao_solar_valor       51696 non-null float64
temperatura_ar_vmedio      51696 non-null float64
umidade_ar_vmedio          51696 non-null float64
velocidade_vento_vmedio    51696 non-null float64
dtypes: float64(7)
memory usage: 5.7 MB
```

In [42]:

```
#CÉLULA TCC-42
#Removendo novamente, pois o resample gera novamente os dias, dias com medições abaixo
 do teto
interDownMete['date'] = interDownMete.index.date # coluna auxiliar
for i in lstDaysOutMete:# correndo todos os dias com falhas nas medições
    idx = interDownMete[interDownMete.date == i].index # obtendo o index
    interDownMete.drop(idx, inplace=True) # removendo do dataset
interDownMete.drop(['date'], axis=1, inplace=True) # removendo coluna auxiliar
```

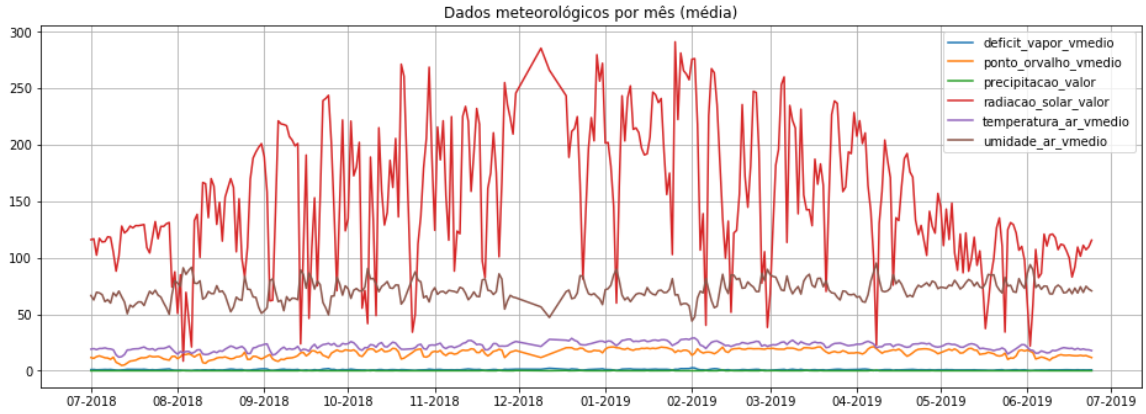
In [43]:

```
#CÉLULA TCC-43
interDownMete.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 48960 entries, 2018-07-01 00:00:00 to 2019-06-24 23:50:00
Data columns (total 7 columns):
deficit_vapor_vmedio       48960 non-null float64
ponto_orvalho_vmedio       48960 non-null float64
precipitacao_valor         48960 non-null float64
radiacao_solar_valor       48960 non-null float64
temperatura_ar_vmedio      48960 non-null float64
umidade_ar_vmedio          48960 non-null float64
velocidade_vento_vmedio    48960 non-null float64
dtypes: float64(7)
memory usage: 3.0 MB
```

In [44]:

```
#CÉLULA TCC-44
plt.figure(figsize=(16,6))
plt.title("Dados meteorológicos por mês (média)")
showDat = interDownMete.groupby(by=interDownMete.index.date).agg({
    'deficit_vapor_vmedio':'mean',
    'ponto_orvalho_vmedio':'mean',
    'precipitacao_valor':'mean',
    'radiacao_solar_valor':'mean',
    'temperatura_ar_vmedio':'mean',
    'umidade_ar_vmedio':'mean'
})
plt.gca().xaxis.set_major_locator(MonthLocator())
plt.gca().xaxis.set_major_formatter(DateFormatter('%m-%Y'))
plt.gcf().autofmt_xdate()
showDataPlot(showDat)
```



1.3.3 - Realizando o merge

Com os datasets padronizados na mesma freqência de medições, é possível agrupar ('merge') os dois em um só dataset, tornando o dataset mais interesssante por conter ambos os registros unificados.

In [45]:

```
#CÉLULA TCC-45
#Realizando merge pelo index, pois os dois datasets estão nesse momento com a mesma fre
quência de medição, onde o mesmo tempo
#existir nos dois, será realizado a junção dos dados em uma mesma linha.
dfSet1 = pd.merge(interDownFoto, interDownMete, left_index=True, right_index=True)
dfSet1.reset_index(inplace=True)
```

In [46]:

```
#CÉLULA TCC-46
dfSet1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 44352 entries, 0 to 44351
Data columns (total 10 columns):
data                        44352 non-null datetime64[ns]
painel                      44352 non-null float64
energia                     44352 non-null float64
deficit_vapor_vmedio        44352 non-null float64
ponto_orvalho_vmedio        44352 non-null float64
precipitacao_valor          44352 non-null float64
radiacao_solar_valor        44352 non-null float64
temperatura_ar_vmedio       44352 non-null float64
umidade_ar_vmedio           44352 non-null float64
velocidade_vento_vmedio     44352 non-null float64
dtypes: datetime64[ns](1), float64(9)
memory usage: 3.4 MB
```

In [47]:

```
#CÉLULA TCC-47
#Salvando o dataset
saveDataFile(dfSet1,'input/FotoMet_P1_DataSets_10T.csv')
```

Com os dados preparados, é possível dar continuidade no trabalho para inicio da geração dos modelos.

In [48]:

```
#CÉLULA TCC-48
#Estrutura respnsável por carregar um dataset no formato csv
#FotoMet_P1_DataSets_10T.csv - 44352 registros - 308 dias
#FotoMet_P2_DataSets_10T.csv - 43920 registros - 305 dias
#FotoMet_P3_DataSets_10T.csv - 43920 registros - 305 dias
#FotoMet_P4_DataSets_10T.csv - 44352 registros - 308 dias
dfSet1 = loadDataFile('input/FotoMet_P1_DataSets_10T.csv', True)
```

In [49]:

```
#CÉLULA TCC-49
allData = dfSet1
allData = allData.groupby(by=allData.index.date).agg({
    'energia':'sum',
    'deficit_vapor_vmedio':'mean',
    'ponto_orvalho_vmedio':'mean',
    'precipitacao_valor':'mean',
    'radiacao_solar_valor':'mean',
    'temperatura_ar_vmedio':'mean',
    'umidade_ar_vmedio':'mean',
    'velocidade_vento_vmedio':'mean'
})
```

In [50]:

```
#CÉLULA TCC-50
#Exibição de um gráfico com dados de energia (acumulada) e umidade do ar (média)
x_values = allData.index
y_values_1 = allData.energia

fig, ax1 = plt.subplots(figsize=(15,7))
title = ('Energia acumulada X Umidade do ar média')

plt.title(title, fontsize=20)
plt.grid(axis='y',color='grey', linestyle='--', lw=0.5, alpha=0.5)
plt.tick_params(axis='both', labelsize=14)

plot1 = ax1.plot(x_values, y_values_1, 'b', label='Energia')
ax1.set_ylabel('Energia (Wh)', fontsize=18)
ax1.set_xlabel('Mês', fontsize=18)
ax1.set_ylim(0,y_values_1.max()+1)
for tl in ax1.get_yticklabels():
    tl.set_color('b')

ax2 = ax1.twinx()
y_values_2 = allData['umidade_ar_vmedio']
plot2 = ax2.plot(x_values, y_values_2, 'r', label='Umidade')

ax2.set_ylabel('Umidade (%)', fontsize=18)
ax2.set_ylim(40, y_values_2.max().max())
ax2.tick_params(axis='y', labelsize=14)
for tl in ax2.get_yticklabels():
    tl.set_color('r')

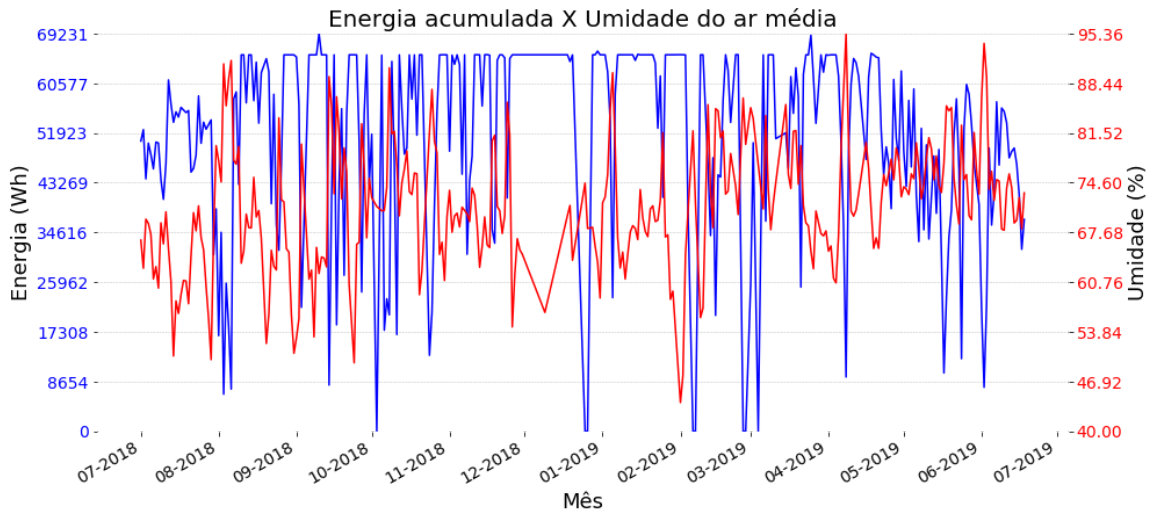
lines = plot1 + plot2

ax1.set_yticks(np.linspace(ax1.get_ybound()[0], ax1.get_ybound()[1], 9))
ax2.set_yticks(np.linspace(ax2.get_ybound()[0], ax2.get_ybound()[1], 9))

for ax in [ax1, ax2]:
    ax.spines["top"].set_visible(False)
    ax.spines["bottom"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.spines["left"].set_visible(False)

plt.gca().xaxis.set_major_locator(MonthLocator())
plt.gca().xaxis.set_major_formatter(DateFormatter('%m-%Y'))
plt.gcf().autofmt_xdate()

plt.show()
```



In [51]:

```
#CÉLULA TCC-51
dfSet1.describe()
#dfSet1['energia'].sum()
#1- 15789723.0 #somatoria da energia em todo o dataset para o painel 1
#2- 11704131.0 #somatoria da energia em todo o dataset para o painel 2
#3- 14946955.0 #somatoria da energia em todo o dataset para o painel 3
#4- 12525875.0 #somatoria da energia em todo o dataset para o painel 4
```

Out[51]:

	painel	energia	deficit_vapor_vmedio	ponto_orvalho_vmedio	precipit
count	44352.0	44352.000000	44352.000000	44352.000000	44352.0
mean	1.0	356.009267	0.884567	15.656145	0.02761
std	0.0	604.252422	0.798869	3.733724	0.28413
min	1.0	0.000000	0.040000	0.500000	0.00000
25%	1.0	0.000000	0.300000	12.800000	0.00000
50%	1.0	0.000000	0.590000	16.300000	0.00000
75%	1.0	576.000000	1.250000	18.700000	0.00000
max	1.0	9297.000000	5.440000	24.600000	11.8000

In [52]:

```
#CÉLULA TCC-52
dfSet1.head()
```

Out[52]:

	painel	energia	deficit_vapor_vmedio	ponto_orvalho_vmedio	precipitacao_
data					
2018-07-01 00:00:00	1.0	0.0	0.33	11.7	0.0
2018-07-01 00:10:00	1.0	0.0	0.31	12.0	0.0
2018-07-01 00:20:00	1.0	0.0	0.32	12.0	0.0
2018-07-01 00:30:00	1.0	0.0	0.31	11.9	0.0
2018-07-01 00:40:00	1.0	0.0	0.29	11.9	0.0

2 - Normalização

Necessário normalizar os dados, pois estão dernormalizados

In [53]:

```
#CÉLULA TCC-53
dfSet1 = dfSet1.reset_index()
df = dfSet1.drop({'data'},axis=1)
```

In [54]:

```
#CÉLULA TCC-54
min_max_scaler = MinMaxScaler(feature_range=(0, 1))
df[df.columns] = min_max_scaler.fit_transform(df[df.columns])
df.describe()
```

Out[54]:

	painel	energia	deficit_vapor_vmedio	ponto_orvalho_vmedio	precipit
count	44352.0	44352.000000	44352.000000	44352.000000	44352.0
mean	0.0	0.038293	0.156401	0.628886	0.00234
std	0.0	0.064994	0.147939	0.154926	0.02407
min	0.0	0.000000	0.000000	0.000000	0.00000
25%	0.0	0.000000	0.048148	0.510373	0.00000
50%	0.0	0.000000	0.101852	0.655602	0.00000
75%	0.0	0.061955	0.224074	0.755187	0.00000
max	0.0	1.000000	1.000000	1.000000	1.00000

3 - Definição do modelo

Nessa etapa, serão utilizado alguns modelos, realizado assim os treinamentos e análisando os resultados obtidos perantes aos dados.

Primeiramente, serão abordados modelos lineares.

As métricas utilizadas a fim da verificação e comparação dos modelos são:

- MSE - Representa o erro médio quadrático, uma métrica de risco correspondente ao valor esperado do erro ou perda quadrática.
- RMSE - Representa o desvio padrão da amostra das diferenças entre os valores previstos e os valores observados (chamados resíduos).
- r² - Coeficiente de Determinação, diz o quanto meu modelo está prevendo corretamente.
- MAE - Representa a média da diferença absoluta entre os valores previstos e o valor observado.

In [55]:

```
#CÉLULA TCC-55
#Imports necessários para análise dos modelos
import seaborn as sns
from math import sqrt

#Imports relacionados aos modelos utilizados
from sklearn import model_selection, linear_model
from sklearn.ensemble import RandomForestRegressor
import xgboost as xgb
from sklearn.model_selection import GridSearchCV
from keras.models import Sequential

#Imports relacionados à métricas
from sklearn import metrics
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

#Import para a normalização
from sklearn.preprocessing import MinMaxScaler

#Import relacionados a rede
from keras.layers import Dense, LSTM, Dropout

import keras as K
import tensorflow as tf
```

```
C:\ProgramData\Anaconda3\lib\site-packages\sklearn\ensemble\weight_boostin
g.py:29: DeprecationWarning:

numpy.core.umath_tests is an internal NumPy module and should not be impor
ted. It will be removed in a future NumPy release.

C:\ProgramData\Anaconda3\lib\site-packages\h5py\__init__.py:36: FutureWarn
ing:

Conversion of the second argument of issubdtype from `float` to `np.floati
ng` is deprecated. In future, it will be treated as `np.float64 == np.dtyp
e(float).type`.

Using TensorFlow backend.
```

In [56]:

```
#CÉLULA TCC-56
#Método responsável por plotar o pair plot
def showPairPlot(dataFrame):

    pairPlot = sns.pairplot(
        dataFrame,
        size=2.0,
        aspect=1.4,
        plot_kws=dict(edgecolor="k", linewidth=0.2),
        diag_kws=dict(shade=True),
        diag_kind="kde",
    )

    fig = pairPlot.fig
    fig.subplots_adjust(top=0.95)
    fig.suptitle(
        'Análise de parâmetros',
        fontsize=13,
        fontweight='bold'
    )
```

In [57]:

```
#CÉLULA TCC-57
#Método responsável por plotar o heatmap
def showHeatMap(dataFrame):

    fig, (ax) = plt.subplots(1, 1, figsize=(11,7))
    heatMap = sns.heatmap(
        dataFrame.corr(),
        ax=ax,
        cmap="coolwarm",
        annot=True,
        fmt='.2f',
        linewidths=.07
    )

    fig.suptitle(
        'Correlação de parâmetros',
        fontsize=13,
        fontweight='bold'
    )
```

In [58]:

```
#CÉLULA TCC-58
#Método responsável por plotar os resultados de treino/testes
def showResultPlot(y_pred,y_test):
    dfteste = pd.DataFrame({'Atual': y_test, 'Previsto': y_pred})
    dfTesteShow = dfteste.iloc[61:131]
    dfTesteShow.plot(kind='bar', figsize=(16,6), title='Exibição de 70 dados de testes
 previstos')
    plt.grid()
    plt.show()
```

In [59]:

```
#CÉLULA TCC-59
#Método responsável por treinar os modelos e exibir as métricas utilizadas para compara
ção dos mesmos
def showResult(model, X_train, y_train, X_test, y_test):

    model.fit(X_train, y_train)
    y_pred_Train = model.predict(X_train)
    y_pred_Test = model.predict(X_test)

    #print(model)# exibição dos parametros utilizados no modelo
    print('------------------- TREINO -------------------------')
    print('MSE: {:.6f}'.format(metrics.mean_squared_error(y_train, y_pred_Train)))
    print('RMSE: {:.6f}'.format(np.sqrt(metrics.mean_squared_error(y_train, y_pred_Trai
n))))
    #print('r²: {:.2f} %'.format(model.score(X_train, y_train) * 100))
    print('r²: {:.2f} %'.format(r2_score(y_train, y_pred_Train) * 100))
    print('MAE: {:.6f}'.format(mean_absolute_error(y_train, y_pred_Train)))
    print('------------------- TESTE --------------------------')
    print('MSE: {:.6f}'.format(metrics.mean_squared_error(y_test, y_pred_Test)))
    print('RMSE: {:.6f}'.format(np.sqrt(metrics.mean_squared_error(y_test, y_pred_Test
))))
    #print('r²: {:.2f} %'.format(model.score(X_test, y_test) * 100))
    print('r²: {:.2f} %'.format(r2_score(y_test, y_pred_Test) * 100))
    print('MAE: {:.6f}'.format(mean_absolute_error(y_test, y_pred_Test)))

    print('----------------------------------------------------')
    showResultPlot(y_pred_Test,y_test)

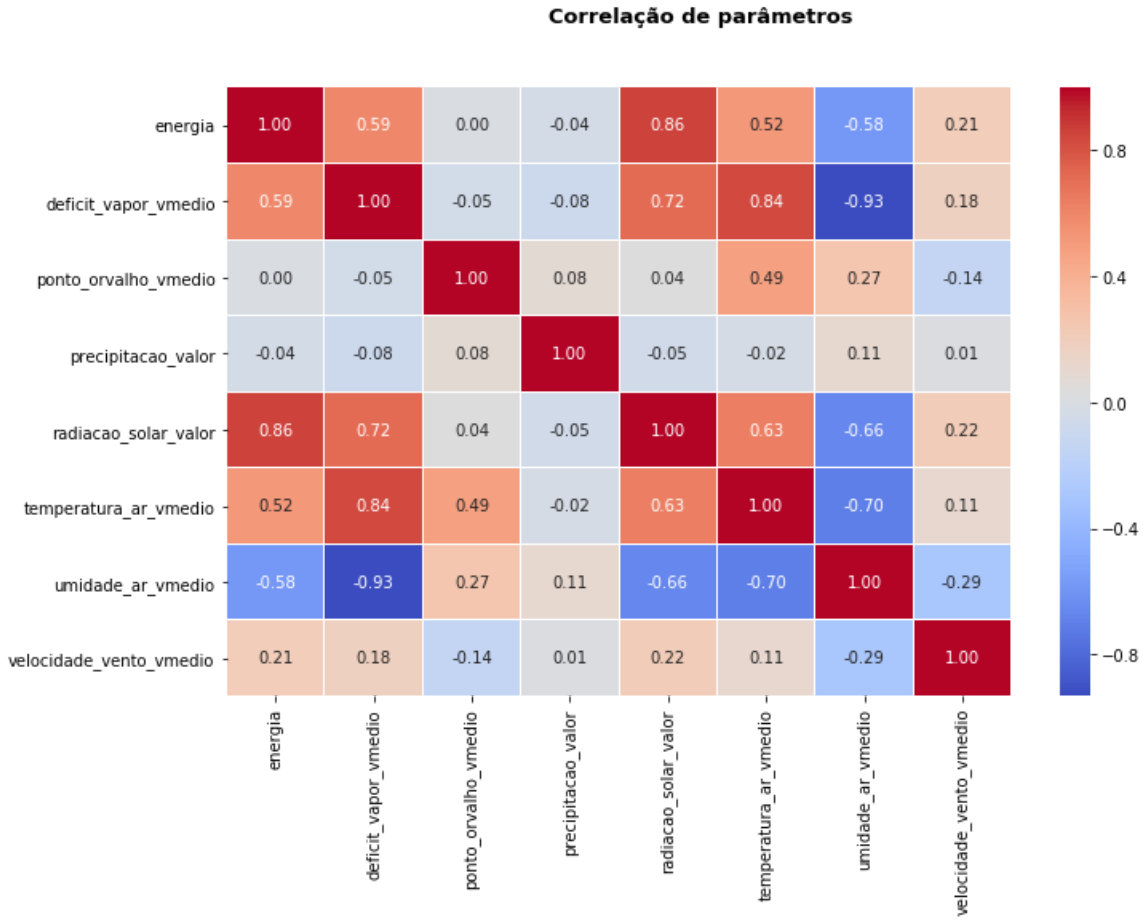
    return model
```

In [60]:

```
#CÉLULA TCC-60
#Excluindo o campo painel, visto que nesse momento consta apenas um.
dfParameters = df.drop(['painel'],axis=1)
```

In [61]:

```
#CÉLULA TCC-61
#Gráfico que demonstra como as colunas estão correlacionadas ao campo de energia
showHeatMap(dfParameters)
```

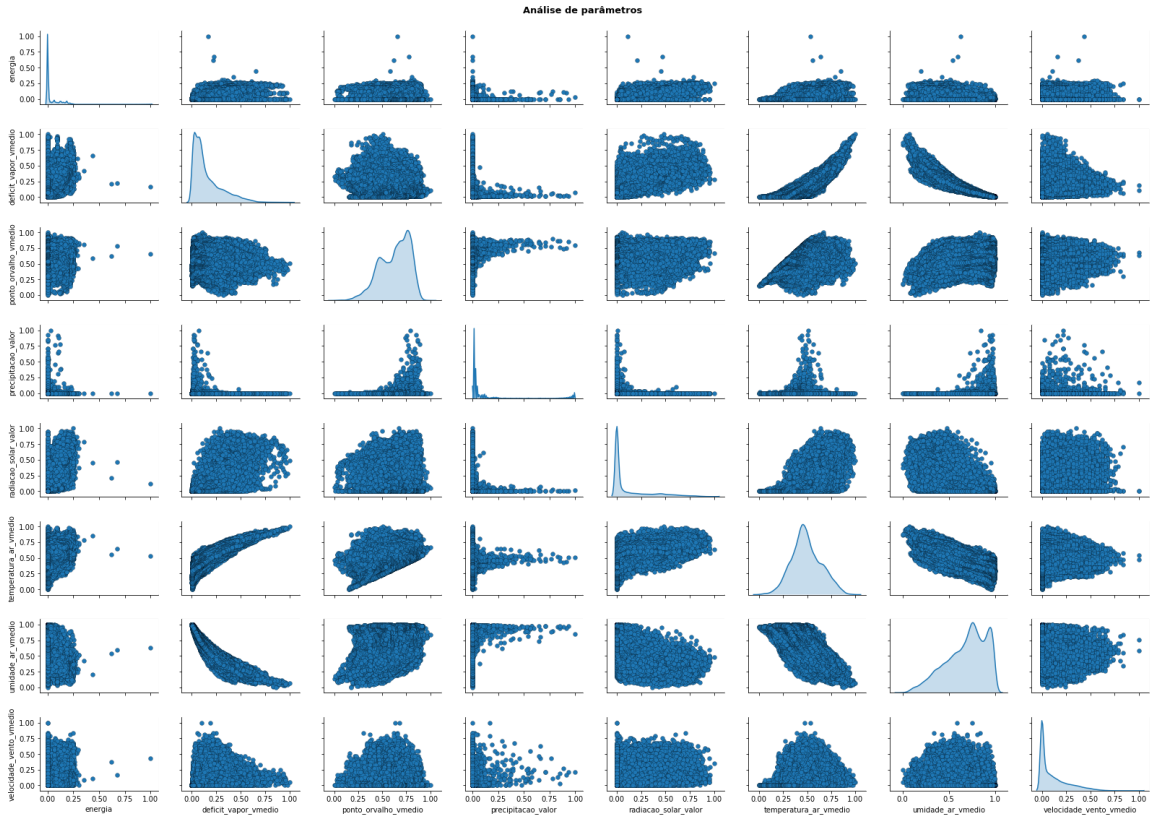


In [62]:

```
#CÉLULA TCC-62
showPairPlot(dfParameters)
```

```
C:\ProgramData\Anaconda3\lib\site-packages\scipy\stats\stats.py:1713: Futu
reWarning:

Using a non-tuple sequence for multidimensional indexing is deprecated; us
e `arr[tuple(seq)]` instead of `arr[seq]`. In the future this will be inte
rpreted as an array index, `arr[np.array(seq)]`, which will result either
in an error or a different result.
```



In [63]:

```
#CÉLULA TCC-63
#Removendo o campo 'energia' do eixo X e definindo o target(y) como o campo 'energia'
X = dfParameters.drop(['energia'],axis=1)
y = dfParameters.energia
```

In [64]:

```
#CÉLULA TCC-64
#Realizando divisão em dados de treino e teste, na proporção de 85% para treino e 15% p
ara testes, de forma contínua
X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size=0.1
5, random_state=42, shuffle=False)
print(X_train.shape, y_train.shape, X_test.shape, y_test.shape)
```

```
(37699, 7) (37699,) (6653, 7) (6653,)
```

Regressão Linear com radiação solar

Regressão linear usa os pontos de dados para encontrar a melhor linha de ajuste para modelar os dados.

In [65]:

```
#CÉLULA TCC-65
#Método responsável por conter a criação do modelo de regressão linear, treinamento e e
xibição dos resultados.
def modelLinearRegression(X_train, y_train, X_test, y_test):
    LinearRegression = linear_model.LinearRegression()

    print('-----------------------------------------------------')
    print('----------      REGRESSÃO LINEAR       -------------')
    print('-----------------------------------------------------')

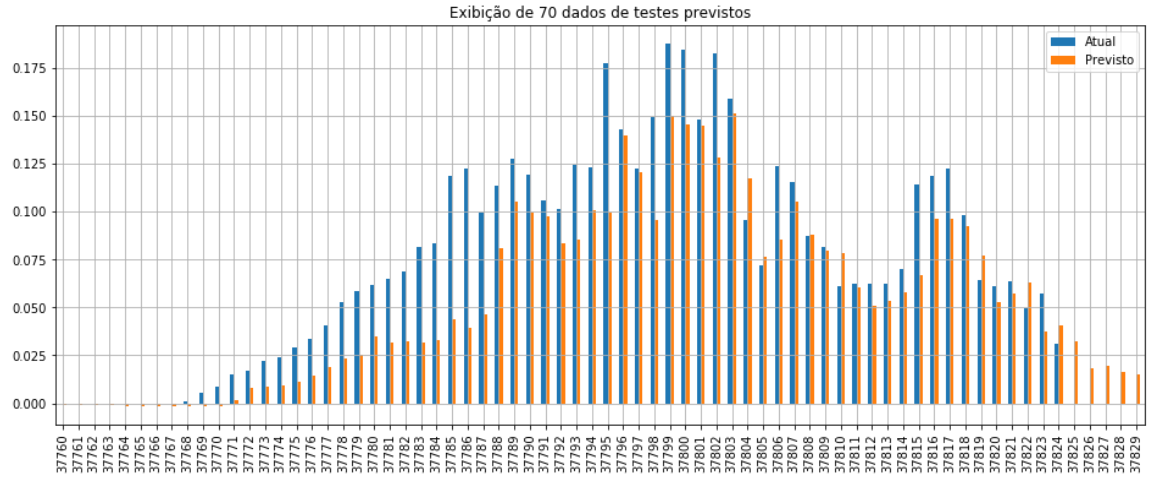
    modelLinearRegression = showResult(LinearRegression, X_train, y_train, X_test, y_te
st)

    coeff = pd.DataFrame(X_train.columns)
    coeff['Coeficiente estimado'] = pd.Series(modelLinearRegression.coef_)
    coeff.sort_values(by='Coeficiente estimado', inplace=True, ascending=False)
    print(coeff)
```

In [66]:

```
#CÉLULA TCC-66
#Execução do modelo de regressão linear
modelLinearRegression(X_train, y_train, X_test, y_test)
```

```
------------------------------------------------------
----------      REGRESSÃO LINEAR       -------------
------------------------------------------------------
------------------- TREINO -------------------------
MSE: 0.001099
RMSE: 0.033155
r²: 75.25 %
MAE: 0.018321
------------------- TESTE --------------------------
MSE: 0.000599
RMSE: 0.024482
r²: 79.59 %
MAE: 0.014412
------------------------------------------------------
```



	0	Coeficiente estimado
3	radiacao_solar_valor	0.281121
2	precipitacao_valor	0.032015
1	ponto_orvalho_vmedio	0.023197
4	temperatura_ar_vmedio	-0.004007
6	velocidade_vento_vmedio	-0.007560
5	umidade_ar_vmedio	-0.105944
0	deficit_vapor_vmedio	-0.159349

Random Forest Regressor com radiação solar

Este algoritmo de florestas aleatórias, gera várias árvores de decisão e a combinação delas é utilizada para se obter uma maior acurácia e uma maior estabilidade.

Encontrando os melhores parâmetros.

In [67]:

```
#CÉLULA TCC-67
#Célula responsável apenas por encontrar os melhores parâmetros para o regressor Random Forest.
rfr = RandomForestRegressor()

parameters = {
    'max_depth': [5, 7, 9, 10],         #número máximo de levels para cada árvore de de cisão
    'max_features': ['sqrt'],           #número de recursos a serem considerados ao pro curar a melhor divisão sqrt(n_features)
    'min_samples_leaf': [7, 9, 10, 12], #número mínimo de amostras necessárias para ser um nó folha
    'min_samples_split': [2, 3, 4],     #número mínimo de amostras necessárias para div idir um nó interno
    'n_estimators': [175, 200, 250]     #número de árvores da floresta
}

rfr_grid = GridSearchCV(
    rfr,
    parameters,
    cv = 2,
    n_jobs = 5,
    verbose=True
)

rfr_grid.fit(X_train,y_train)

print(rfr_grid.best_score_)
print(rfr_grid.best_params_)
```

```
Fitting 2 folds for each of 144 candidates, totalling 288 fits

[Parallel(n_jobs=5)]: Done  40 tasks      | elapsed:  1.4min
[Parallel(n_jobs=5)]: Done 190 tasks      | elapsed:  4.3min
[Parallel(n_jobs=5)]: Done 288 out of 288 | elapsed:  6.7min finished

0.7300996957998043
{'max_depth': 9, 'max_features': 'sqrt', 'min_samples_leaf': 12, 'min_samp
les_split': 2, 'n_estimators': 200}
```

In [68]:

```
#CÉLULA TCC-68
#Método responsável por conter a criação do modelo de regressão Random Forest, treiname
nto e exibição dos resultados.
def modelRandomRorest(X_train, y_train, X_test, y_test):
    RandomForest = RandomForestRegressor(
        max_depth = 9,
        max_features = 'sqrt',
        min_samples_leaf = 12,
        min_samples_split = 2,
        n_estimators = 200
    )

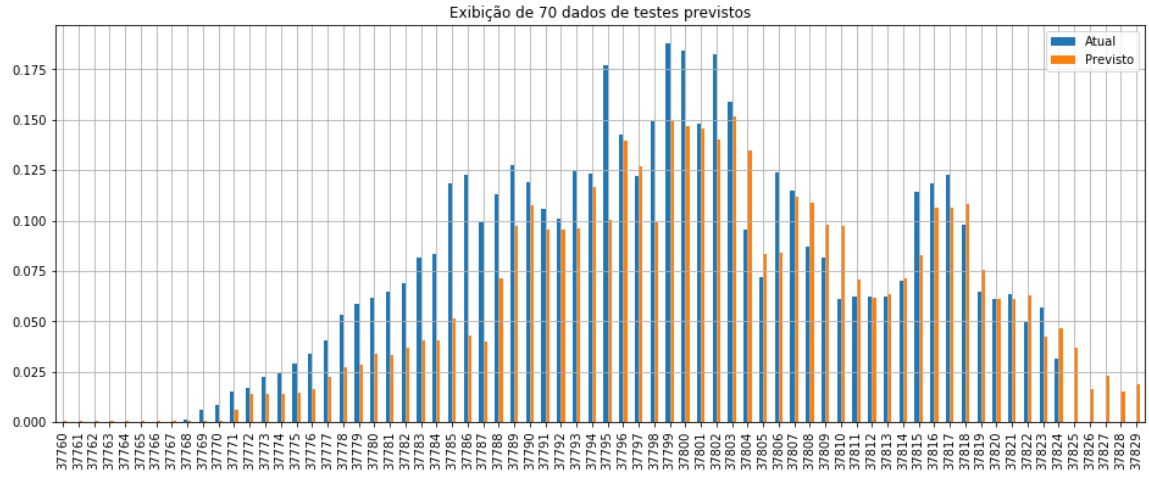
    print('-----------------------------------------------------')
    print('------------      RANDOM FOREST     ----------------')
    print('-----------------------------------------------------')

    modelRandomForest = showResult(RandomForest, X_train, y_train, X_test, y_test)
```

In [69]:

```
#CÉLULA TCC-69
#Execução do modelo de regressão random forest
modelRandomRorest(X_train, y_train, X_test, y_test)
```

```
-----------------------------------------------------
------------      RANDOM FOREST     ----------------
-----------------------------------------------------
------------------- TREINO --------------------------
MSE: 0.000865
RMSE: 0.029414
r²: 80.52 %
MAE: 0.015189
------------------- TESTE ---------------------------
MSE: 0.000506
RMSE: 0.022502
r²: 82.76 %
MAE: 0.011234
-----------------------------------------------------
```



XGBRegressor com radiação solar

É uma implementação de árvores de decisão aprimoradas por gradiente.

Encontrando os melhores parâmetros.

In [70]:

```
#CÉLULA TCC-70
#Célula responsável apenas por encontrar os melhores parâmetros para o regressor XGB.
xgbReg = xgb.XGBRegressor()

parameters = {
    'colsample_bytree': [0.6, 0.7],      #proporção de subamostra de colunas ao constru
ir cada árvore
    'learning_rate': [0.03],             #taxa de aprendizagem
    'max_depth': [3, 4, 5, 6],           #profundidade máxima da árvore para os aprendi
zes
    'min_child_weight': [6, 7, 8, 9],    #soma mínima do peso da instância (hessian) ne
cessária em um filho
    'n_estimators': [150, 175, 225, 250],#número de árvores no treinamento
    'n_jobs ':[4],                       #número de threads paralelas usadas para rodar
xgboost
    'objective':['reg:squarederror'],    #especifique a tarefa de aprendizagem e o obje
tivo de aprendizagem correspondente
    'subsample': [0.7, 0.8]              #proporção de subamostra da instância de trein
amento
}

xgb_grid = GridSearchCV(
    xgbReg,
    parameters,
    cv = 2,
    n_jobs = 5,
    verbose=True
)

xgb_grid.fit(X_train,y_train)

print(xgb_grid.best_score_)
print(xgb_grid.best_params_)
```

```
Fitting 2 folds for each of 256 candidates, totalling 512 fits

[Parallel(n_jobs=5)]: Done  40 tasks      | elapsed:  1.3min
[Parallel(n_jobs=5)]: Done 190 tasks      | elapsed:  4.8min
[Parallel(n_jobs=5)]: Done 440 tasks      | elapsed: 11.5min
[Parallel(n_jobs=5)]: Done 512 out of 512 | elapsed: 13.6min finished
C:\ProgramData\Anaconda3\lib\site-packages\xgboost\core.py:587: FutureWarn
ing:

Series.base is deprecated and will be removed in a future version


0.7298077367843167
{'colsample_bytree': 0.6, 'learning_rate': 0.03, 'max_depth': 3, 'min_chil
d_weight': 8, 'n_estimators': 175, 'n_jobs ': 4, 'objective': 'reg:squared
error', 'subsample': 0.7}
```

In [71]:

```
#CÉLULA TCC-71
#Método responsável por conter a criação do modelo de regressão XGB, treinamento e exib
ição dos resultados.
def modelXgbRegressor(X_train, y_train, X_test, y_test):
    xgbReg = xgb.XGBRegressor(
        colsample_bytree = 0.6,
        learning_rate = 0.03,
        max_depth = 3,
        min_child_weight = 8,
        n_estimators = 175,
        n_jobs = 4,
        objective = 'reg:squarederror',
        subsample = 0.7
    )

    print('-----------------------------------------------------')
    print('------------           XGB          ----------------')
    print('-----------------------------------------------------')

    modelRandomForest = showResult(xgbReg, X_train, y_train, X_test, y_test)
```

In [72]:

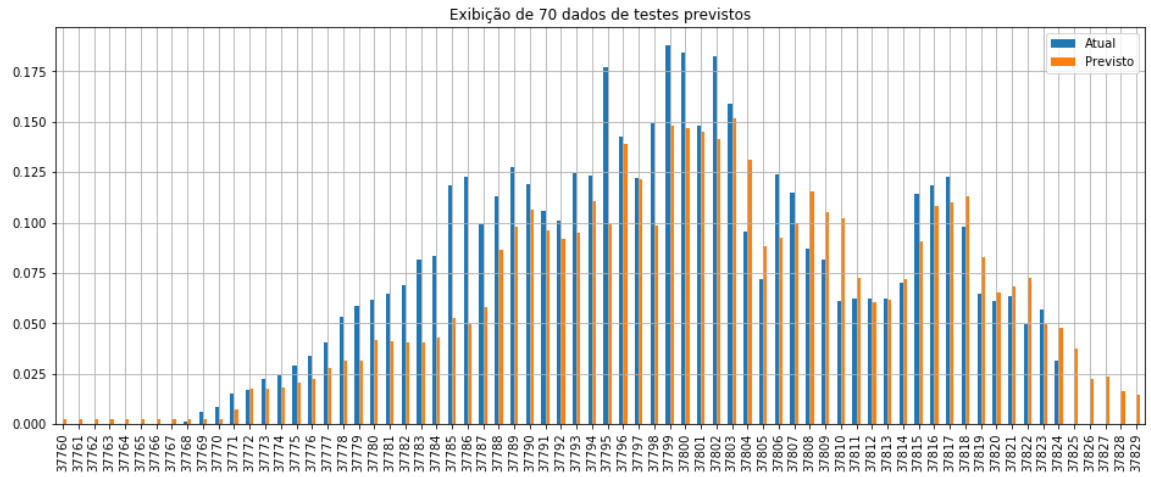
```
#CÉLULA TCC-72
#Execução do modelo de regressão XGB
modelXgbRegressor(X_train, y_train, X_test, y_test)
```

```
------------------------------------------------------
------------           XGB          ----------------
------------------------------------------------------

C:\ProgramData\Anaconda3\lib\site-packages\xgboost\core.py:587: FutureWarn
ing:

Series.base is deprecated and will be removed in a future version


------------------- TREINO -------------------------
MSE: 0.000973
RMSE: 0.031200
r²: 78.08 %
MAE: 0.017253
------------------- TESTE --------------------------
MSE: 0.000519
RMSE: 0.022789
r²: 82.32 %
MAE: 0.012499
------------------------------------------------------
```



Execução dos modelos sem o campo 'radiação_solar_valor'

Visto que o campo referente a radiação solar está altamente relacionado a geração de energia (conforme o gráfico da CÉLULA TCC-61), optou-se por refazer os modelos sem a existência do mesmo, para verificação dos resultados.

In [73]:

```
#CÉLULA TCC-73
#Removendo o campo 'energia' e 'radiacao_solar_valor' do eixo X e definindo o target(y) como o campo 'energia'
X = dfParameters.drop({'energia','radiacao_solar_valor'},axis=1)
y = dfParameters.energia
```

In [74]:

```
#CÉLULA TCC-74
#Realizando divisão em dados de treino e teste, na proporção de 85% para treino e 15% para testes, de forma continua
X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size=0.15, random_state=42, shuffle=False)
print(X_train.shape, y_train.shape, X_test.shape, y_test.shape)
```

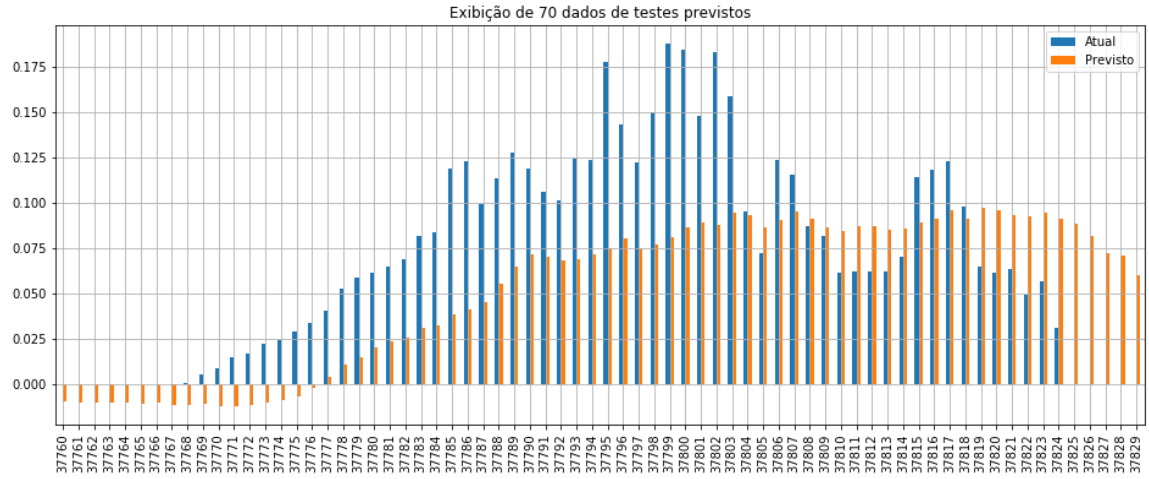
```
(37699, 6) (37699,) (6653, 6) (6653,)
```

Regressão Linear

In [75]:

```
#CÉLULA TCC-75
modelLinearRegression(X_train, y_train, X_test, y_test)
```

```
------------------------------------------------------
----------      REGRESSÃO LINEAR       -------------
------------------------------------------------------
------------------- TREINO --------------------------
MSE: 0.002788
RMSE: 0.052797
r²: 37.23 %
MAE: 0.038524
------------------- TESTE ---------------------------
MSE: 0.001858
RMSE: 0.043104
r²: 36.74 %
MAE: 0.031524
------------------------------------------------------
```



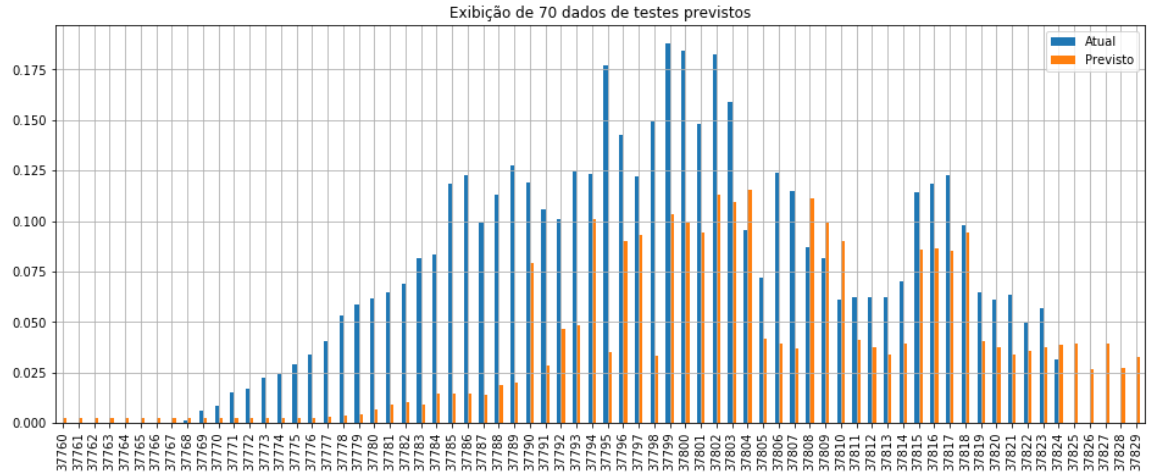
	0	Coeficiente estimado
0	deficit_vapor_vmedio	0.242796
1	ponto_orvalho_vmedio	0.215840
2	precipitacao_valor	0.034931
5	velocidade_vento_vmedio	0.034596
4	umidade_ar_vmedio	-0.183731
3	temperatura_ar_vmedio	-0.260302

Random Forest Regressor

In [76]:

```
#CÉLULA TCC-76
modelRandomRorest(X_train, y_train, X_test, y_test)
```

```
------------------------------------------------------
------------      RANDOM FOREST     -----------------
------------------------------------------------------
------------------- TREINO -------------------------
MSE: 0.002089
RMSE: 0.045708
r²: 52.96 %
MAE: 0.029382
------------------- TESTE ---------------------------
MSE: 0.001582
RMSE: 0.039779
r²: 46.13 %
MAE: 0.023850
------------------------------------------------------
```



XGBRegressor

In [77]:

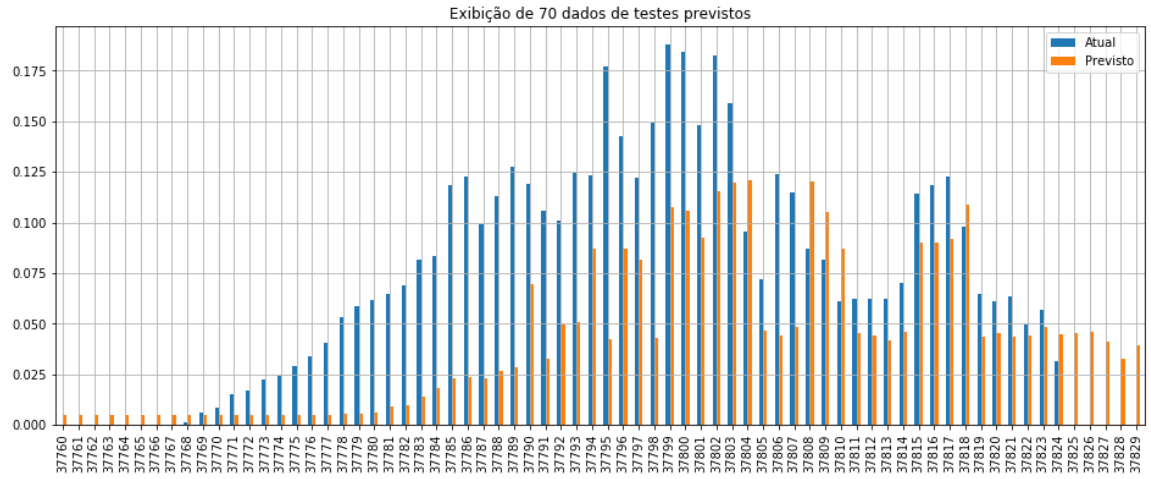
```
#CÉLULA TCC-77
modelXgbRegressor(X_train, y_train, X_test, y_test)
```

```
------------------------------------------------------
------------           XGB          -----------------
------------------------------------------------------

C:\ProgramData\Anaconda3\lib\site-packages\xgboost\core.py:587: FutureWarn
ing:

Series.base is deprecated and will be removed in a future version


------------------- TREINO -------------------------
MSE: 0.002295
RMSE: 0.047907
r²: 48.32 %
MAE: 0.032339
------------------- TESTE --------------------------
MSE: 0.001515
RMSE: 0.038922
r²: 48.42 %
MAE: 0.024991
------------------------------------------------------
```



Uma outra abordagem, realizando o uso de rede neural, considerando os dados como uma série temporal.

Long Short Term Memory

É uma arquitetura de rede neural recorrente.

In [78]:

```
#CÉLULA TCC-78
#Convertendo series para aprendizado supervisionado
def series_to_supervised(data, n_in=1, n_out=1, dropnan=True):
    n_vars = 1 if type(data) is list else data.shape[1]
    df = pd.DataFrame(data)
    cols, names = list(), list()
    # input sequence (t-n, ... t-1)
    for i in range(n_in, 0, -1):
        cols.append(df.shift(i))
        names += [('var%d(t-%d)' % (j+1, i)) for j in range(n_vars)]
    # forecast sequence (t, t+1, ... t+n)
    for i in range(0, n_out):
        cols.append(df.shift(-i))
        if i == 0:
            names += [('var%d(t)' % (j+1)) for j in range(n_vars)]
        else:
            names += [('var%d(t+%d)' % (j+1, i)) for j in range(n_vars)]
    agg = pd.concat(cols, axis=1)
    agg.columns = names
    if dropnan:
        agg.dropna(inplace=True)
    return agg
```

In [91]:

```
#CÉLULA TCC-79
#Carregando novamente o csv
df = loadDataFile('input/FotoMet_P1_DataSets_10T.csv', True)
df.drop({'painel','radiacao_solar_valor'}, axis=1, inplace=True)
```

In [92]:

```
#CÉLULA TCC-80
#Normalizando os dados
min_max_scaler = MinMaxScaler(feature_range=(0, 1))
scalerDf = min_max_scaler.fit_transform(df.values.astype('float32'))
```

In [93]:

```
#CÉLULA TCC-81
#Convertendo serie para um modelo supervisionado
newDf = series_to_supervised(scalerDf, 1, 1)

#Removendo os campos que não serão previstos, deixando somente o campo referente a ener
gia, neste caso var1(t),
#ou seja, a previsão da próxima.
newDf.drop(newDf.columns[np.arange(df.shape[1] + 1,newDf.shape[1])], axis=1, inplace=Tr
ue)
print(newDf.head())
```

```
   var1(t-1)  var2(t-1)  var3(t-1)  var4(t-1)  var5(t-1)  var6(t-1)  \
1        0.0   0.053704   0.464730        0.0   0.298100   0.796529
2        0.0   0.050000   0.477178        0.0   0.298990   0.813269
3        0.0   0.051852   0.477178        0.0   0.298100   0.810807
4        0.0   0.050000   0.473029        0.0   0.294240   0.814131
5        0.0   0.046296   0.473029        0.0   0.289786   0.825455

   var7(t-1)  var1(t)
1        0.0      0.0
2        0.0      0.0
3        0.0      0.0
4        0.0      0.0
5        0.0      0.0
```

In [94]:

```
#CÉLULA TCC-82
# Dividindo os dados em treino e teste
values = newDf.values
n_division = int(len(values) * 0.85)
train = values[:n_division, :]
test = values[n_division:, :]

# Realizando split das entradas e saída
X_train, y_train = train[:, 1:-1], train[:, -1]#Removendo energia do eixo X, deixando a
penas a próxima energia no campo y
X_test, y_test = test[:, 1:-1], test[:, -1]

# Realizando o reshap da entrada para ser 3D para a LSTM [samples, timesteps, features]
X_train = X_train.reshape((X_train.shape[0], 1, X_train.shape[1]))
X_test = X_test.reshape((X_test.shape[0], 1, X_test.shape[1]))

print(X_train.shape, y_train.shape, X_test.shape, y_test.shape)
```

```
(37698, 1, 6) (37698,) (6653, 1, 6) (6653,)
```

In [95]:

```
#CÉLULA TCC-83
#Definindo a rede

#Limpa o modelo previamente usado
K.backend.clear_session()
tf.logging.set_verbosity(tf.logging.WARN) #desliga os warnings do tensorflow

model = Sequential()
model.add(LSTM(50, input_shape=(X_train.shape[1], X_train.shape[2])))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam', metrics=['mae', 'mse'])
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #   
=================================================================
lstm_1 (LSTM)                (None, 50)                11400     
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 51        
=================================================================
Total params: 11,451
Trainable params: 11,451
Non-trainable params: 0
_________________________________________________________________
```

In [96]:

```
#CÉLULA TCC-84
#Treinando a rede
max_epochs = 70
batch_size = 60
print("Iniciando treinamento... ")

h = model.fit(X_train, y_train, epochs=max_epochs, batch_size=batch_size, validation_da
ta=(X_test, y_test), verbose=1, shuffle=False)

print("Treinamento finalizado \n")
```

```
Iniciando treinamento...
Train on 37698 samples, validate on 6653 samples
Epoch 1/70
37698/37698 [==============================] - 3s 79us/step - loss: 0.0030
- mean_absolute_error: 0.0405 - mean_squared_error: 0.0030 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0347 - val_mean_squared_error: 0.0020
Epoch 2/70
37698/37698 [==============================] - 2s 53us/step - loss: 0.0029
- mean_absolute_error: 0.0401 - mean_squared_error: 0.0029 - val_loss: 0.0
021 - val_mean_absolute_error: 0.0351 - val_mean_squared_error: 0.0021
Epoch 3/70
37698/37698 [==============================] - 2s 54us/step - loss: 0.0029
- mean_absolute_error: 0.0401 - mean_squared_error: 0.0029 - val_loss: 0.0
021 - val_mean_absolute_error: 0.0351 - val_mean_squared_error: 0.0021
Epoch 4/70
37698/37698 [==============================] - 2s 54us/step - loss: 0.0029
- mean_absolute_error: 0.0400 - mean_squared_error: 0.0029 - val_loss: 0.0
021 - val_mean_absolute_error: 0.0352 - val_mean_squared_error: 0.0021
Epoch 5/70
37698/37698 [==============================] - 2s 63us/step - loss: 0.0029
- mean_absolute_error: 0.0399 - mean_squared_error: 0.0029 - val_loss: 0.0
022 - val_mean_absolute_error: 0.0352 - val_mean_squared_error: 0.0022
Epoch 6/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0029
- mean_absolute_error: 0.0399 - mean_squared_error: 0.0029 - val_loss: 0.0
022 - val_mean_absolute_error: 0.0351 - val_mean_squared_error: 0.0022
Epoch 7/70
37698/37698 [==============================] - 2s 58us/step - loss: 0.0029
- mean_absolute_error: 0.0398 - mean_squared_error: 0.0029 - val_loss: 0.0
022 - val_mean_absolute_error: 0.0351 - val_mean_squared_error: 0.0022
Epoch 8/70
37698/37698 [==============================] - 2s 62us/step - loss: 0.0029
- mean_absolute_error: 0.0397 - mean_squared_error: 0.0029 - val_loss: 0.0
022 - val_mean_absolute_error: 0.0350 - val_mean_squared_error: 0.0022
Epoch 9/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0029
- mean_absolute_error: 0.0396 - mean_squared_error: 0.0029 - val_loss: 0.0
022 - val_mean_absolute_error: 0.0350 - val_mean_squared_error: 0.0022
Epoch 10/70
37698/37698 [==============================] - 2s 56us/step - loss: 0.0029
- mean_absolute_error: 0.0395 - mean_squared_error: 0.0029 - val_loss: 0.0
022 - val_mean_absolute_error: 0.0349 - val_mean_squared_error: 0.0022
Epoch 11/70
37698/37698 [==============================] - 2s 58us/step - loss: 0.0029
- mean_absolute_error: 0.0394 - mean_squared_error: 0.0029 - val_loss: 0.0
022 - val_mean_absolute_error: 0.0348 - val_mean_squared_error: 0.0022
Epoch 12/70
37698/37698 [==============================] - 2s 54us/step - loss: 0.0029
- mean_absolute_error: 0.0393 - mean_squared_error: 0.0029 - val_loss: 0.0
022 - val_mean_absolute_error: 0.0346 - val_mean_squared_error: 0.0022
Epoch 13/70
37698/37698 [==============================] - 2s 53us/step - loss: 0.0029
- mean_absolute_error: 0.0392 - mean_squared_error: 0.0029 - val_loss: 0.0
022 - val_mean_absolute_error: 0.0345 - val_mean_squared_error: 0.0022
Epoch 14/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0029
- mean_absolute_error: 0.0391 - mean_squared_error: 0.0029 - val_loss: 0.0
022 - val_mean_absolute_error: 0.0343 - val_mean_squared_error: 0.0022
Epoch 15/70
37698/37698 [==============================] - 2s 64us/step - loss: 0.0029
- mean_absolute_error: 0.0390 - mean_squared_error: 0.0029 - val_loss: 0.0
```

```
022 - val_mean_absolute_error: 0.0341 - val_mean_squared_error: 0.0022
Epoch 16/70
37698/37698 [==============================] - 2s 61us/step - loss: 0.0028
- mean_absolute_error: 0.0389 - mean_squared_error: 0.0028 - val_loss: 0.0
021 - val_mean_absolute_error: 0.0339 - val_mean_squared_error: 0.0021
Epoch 17/70
37698/37698 [==============================] - 2s 59us/step - loss: 0.0028
- mean_absolute_error: 0.0388 - mean_squared_error: 0.0028 - val_loss: 0.0
021 - val_mean_absolute_error: 0.0337 - val_mean_squared_error: 0.0021
Epoch 18/70
37698/37698 [==============================] - 2s 61us/step - loss: 0.0028
- mean_absolute_error: 0.0386 - mean_squared_error: 0.0028 - val_loss: 0.0
021 - val_mean_absolute_error: 0.0334 - val_mean_squared_error: 0.0021
Epoch 19/70
37698/37698 [==============================] - 2s 48us/step - loss: 0.0028
- mean_absolute_error: 0.0385 - mean_squared_error: 0.0028 - val_loss: 0.0
021 - val_mean_absolute_error: 0.0332 - val_mean_squared_error: 0.0021
Epoch 20/70
37698/37698 [==============================] - 2s 59us/step - loss: 0.0028
- mean_absolute_error: 0.0384 - mean_squared_error: 0.0028 - val_loss: 0.0
021 - val_mean_absolute_error: 0.0329 - val_mean_squared_error: 0.0021
Epoch 21/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0028
- mean_absolute_error: 0.0383 - mean_squared_error: 0.0028 - val_loss: 0.0
021 - val_mean_absolute_error: 0.0327 - val_mean_squared_error: 0.0021
Epoch 22/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0028
- mean_absolute_error: 0.0381 - mean_squared_error: 0.0028 - val_loss: 0.0
021 - val_mean_absolute_error: 0.0325 - val_mean_squared_error: 0.0021
Epoch 23/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0028
- mean_absolute_error: 0.0380 - mean_squared_error: 0.0028 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0323 - val_mean_squared_error: 0.0020
Epoch 24/70
37698/37698 [==============================] - 2s 56us/step - loss: 0.0028
- mean_absolute_error: 0.0379 - mean_squared_error: 0.0028 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0321 - val_mean_squared_error: 0.0020
Epoch 25/70
37698/37698 [==============================] - 2s 55us/step - loss: 0.0028
- mean_absolute_error: 0.0378 - mean_squared_error: 0.0028 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0319 - val_mean_squared_error: 0.0020
Epoch 26/70
37698/37698 [==============================] - 2s 55us/step - loss: 0.0028
- mean_absolute_error: 0.0378 - mean_squared_error: 0.0028 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0318 - val_mean_squared_error: 0.0020
Epoch 27/70
37698/37698 [==============================] - 2s 55us/step - loss: 0.0028
- mean_absolute_error: 0.0377 - mean_squared_error: 0.0028 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0316 - val_mean_squared_error: 0.0020
Epoch 28/70
37698/37698 [==============================] - 2s 54us/step - loss: 0.0028
- mean_absolute_error: 0.0376 - mean_squared_error: 0.0028 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0315 - val_mean_squared_error: 0.0020
Epoch 29/70
37698/37698 [==============================] - 2s 59us/step - loss: 0.0028
- mean_absolute_error: 0.0375 - mean_squared_error: 0.0028 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0314 - val_mean_squared_error: 0.0020
Epoch 30/70
37698/37698 [==============================] - 2s 56us/step - loss: 0.0028
- mean_absolute_error: 0.0375 - mean_squared_error: 0.0028 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0313 - val_mean_squared_error: 0.0020
```

```
Epoch 31/70
37698/37698 [==============================] - 2s 59us/step - loss: 0.0028
- mean_absolute_error: 0.0374 - mean_squared_error: 0.0028 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0312 - val_mean_squared_error: 0.0020
Epoch 32/70
37698/37698 [==============================] - 2s 62us/step - loss: 0.0028
- mean_absolute_error: 0.0373 - mean_squared_error: 0.0028 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0311 - val_mean_squared_error: 0.0020
Epoch 33/70
37698/37698 [==============================] - 2s 60us/step - loss: 0.0028
- mean_absolute_error: 0.0373 - mean_squared_error: 0.0028 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0310 - val_mean_squared_error: 0.0020
Epoch 34/70
37698/37698 [==============================] - 2s 58us/step - loss: 0.0027
- mean_absolute_error: 0.0372 - mean_squared_error: 0.0027 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0309 - val_mean_squared_error: 0.0020
Epoch 35/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0027
- mean_absolute_error: 0.0372 - mean_squared_error: 0.0027 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0308 - val_mean_squared_error: 0.0020
Epoch 36/70
37698/37698 [==============================] - 2s 54us/step - loss: 0.0027
- mean_absolute_error: 0.0371 - mean_squared_error: 0.0027 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0307 - val_mean_squared_error: 0.0020
Epoch 37/70
37698/37698 [==============================] - 2s 53us/step - loss: 0.0027
- mean_absolute_error: 0.0371 - mean_squared_error: 0.0027 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0306 - val_mean_squared_error: 0.0020
Epoch 38/70
37698/37698 [==============================] - 2s 54us/step - loss: 0.0027
- mean_absolute_error: 0.0370 - mean_squared_error: 0.0027 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0305 - val_mean_squared_error: 0.0020
Epoch 39/70
37698/37698 [==============================] - 2s 54us/step - loss: 0.0027
- mean_absolute_error: 0.0370 - mean_squared_error: 0.0027 - val_loss: 0.0
020 - val_mean_absolute_error: 0.0304 - val_mean_squared_error: 0.0020
Epoch 40/70
37698/37698 [==============================] - 2s 54us/step - loss: 0.0027
- mean_absolute_error: 0.0369 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0303 - val_mean_squared_error: 0.0019
Epoch 41/70
37698/37698 [==============================] - 2s 54us/step - loss: 0.0027
- mean_absolute_error: 0.0369 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0302 - val_mean_squared_error: 0.0019
Epoch 42/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0027
- mean_absolute_error: 0.0368 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0301 - val_mean_squared_error: 0.0019
Epoch 43/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0027
- mean_absolute_error: 0.0368 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0300 - val_mean_squared_error: 0.0019
Epoch 44/70
37698/37698 [==============================] - 2s 56us/step - loss: 0.0027
- mean_absolute_error: 0.0367 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0299 - val_mean_squared_error: 0.0019
Epoch 45/70
37698/37698 [==============================] - 3s 67us/step - loss: 0.0027
- mean_absolute_error: 0.0367 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0298 - val_mean_squared_error: 0.0019
Epoch 46/70
```

```
37698/37698 [==============================] - 2s 55us/step - loss: 0.0027
- mean_absolute_error: 0.0366 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0297 - val_mean_squared_error: 0.0019
Epoch 47/70
37698/37698 [==============================] - 2s 58us/step - loss: 0.0027
- mean_absolute_error: 0.0366 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0296 - val_mean_squared_error: 0.0019
Epoch 48/70
37698/37698 [==============================] - 2s 60us/step - loss: 0.0027
- mean_absolute_error: 0.0365 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0295 - val_mean_squared_error: 0.0019
Epoch 49/70
37698/37698 [==============================] - 2s 56us/step - loss: 0.0027
- mean_absolute_error: 0.0364 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0294 - val_mean_squared_error: 0.0019
Epoch 50/70
37698/37698 [==============================] - 2s 55us/step - loss: 0.0027
- mean_absolute_error: 0.0363 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0292 - val_mean_squared_error: 0.0019
Epoch 51/70
37698/37698 [==============================] - 2s 53us/step - loss: 0.0027
- mean_absolute_error: 0.0362 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0291 - val_mean_squared_error: 0.0019
Epoch 52/70
37698/37698 [==============================] - 2s 53us/step - loss: 0.0027
- mean_absolute_error: 0.0361 - mean_squared_error: 0.0027 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0289 - val_mean_squared_error: 0.0019
Epoch 53/70
37698/37698 [==============================] - 2s 55us/step - loss: 0.0026
- mean_absolute_error: 0.0360 - mean_squared_error: 0.0026 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0288 - val_mean_squared_error: 0.0019
Epoch 54/70
37698/37698 [==============================] - 2s 54us/step - loss: 0.0026
- mean_absolute_error: 0.0359 - mean_squared_error: 0.0026 - val_loss: 0.0
019 - val_mean_absolute_error: 0.0286 - val_mean_squared_error: 0.0019
Epoch 55/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0026
- mean_absolute_error: 0.0358 - mean_squared_error: 0.0026 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0285 - val_mean_squared_error: 0.0018
Epoch 56/70
37698/37698 [==============================] - 2s 65us/step - loss: 0.0026
- mean_absolute_error: 0.0357 - mean_squared_error: 0.0026 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0283 - val_mean_squared_error: 0.0018
Epoch 57/70
37698/37698 [==============================] - 2s 64us/step - loss: 0.0026
- mean_absolute_error: 0.0356 - mean_squared_error: 0.0026 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0282 - val_mean_squared_error: 0.0018
Epoch 58/70
37698/37698 [==============================] - 2s 59us/step - loss: 0.0026
- mean_absolute_error: 0.0355 - mean_squared_error: 0.0026 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0281 - val_mean_squared_error: 0.0018
Epoch 59/70
37698/37698 [==============================] - 2s 61us/step - loss: 0.0026
- mean_absolute_error: 0.0354 - mean_squared_error: 0.0026 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0279 - val_mean_squared_error: 0.0018
Epoch 60/70
37698/37698 [==============================] - 2s 59us/step - loss: 0.0026
- mean_absolute_error: 0.0353 - mean_squared_error: 0.0026 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0278 - val_mean_squared_error: 0.0018
Epoch 61/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0026
```

```
- mean_absolute_error: 0.0352 - mean_squared_error: 0.0026 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0276 - val_mean_squared_error: 0.0018
Epoch 62/70
37698/37698 [==============================] - 2s 57us/step - loss: 0.0026
- mean_absolute_error: 0.0352 - mean_squared_error: 0.0026 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0275 - val_mean_squared_error: 0.0018
Epoch 63/70
37698/37698 [==============================] - 2s 58us/step - loss: 0.0026
- mean_absolute_error: 0.0351 - mean_squared_error: 0.0026 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0274 - val_mean_squared_error: 0.0018
Epoch 64/70
37698/37698 [==============================] - 2s 63us/step - loss: 0.0026
- mean_absolute_error: 0.0350 - mean_squared_error: 0.0026 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0273 - val_mean_squared_error: 0.0018
Epoch 65/70
37698/37698 [==============================] - 2s 61us/step - loss: 0.0026
- mean_absolute_error: 0.0350 - mean_squared_error: 0.0026 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0272 - val_mean_squared_error: 0.0018
Epoch 66/70
37698/37698 [==============================] - 2s 59us/step - loss: 0.0025
- mean_absolute_error: 0.0349 - mean_squared_error: 0.0025 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0272 - val_mean_squared_error: 0.0018
Epoch 67/70
37698/37698 [==============================] - 2s 59us/step - loss: 0.0025
- mean_absolute_error: 0.0349 - mean_squared_error: 0.0025 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0271 - val_mean_squared_error: 0.0018
Epoch 68/70
37698/37698 [==============================] - 2s 58us/step - loss: 0.0025
- mean_absolute_error: 0.0348 - mean_squared_error: 0.0025 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0271 - val_mean_squared_error: 0.0018
Epoch 69/70
37698/37698 [==============================] - 2s 60us/step - loss: 0.0025
- mean_absolute_error: 0.0348 - mean_squared_error: 0.0025 - val_loss: 0.0
018 - val_mean_absolute_error: 0.0270 - val_mean_squared_error: 0.0018
Epoch 70/70
37698/37698 [==============================] - 2s 59us/step - loss: 0.0025
- mean_absolute_error: 0.0348 - mean_squared_error: 0.0025 - val_loss: 0.0
017 - val_mean_absolute_error: 0.0270 - val_mean_squared_error: 0.0017
Treinamento finalizado
```

In [97]:

```
#CÉLULA TCC-85
#Avaliando o modelo
eval = model.evaluate(X_train,y_train,verbose=0)
print("Dados de treino: loss = {:.6f} MAE = {:.2f} %".format(eval[0], eval[1]*100))

eval = model.evaluate(X_test,y_test,verbose=0)
print("Dados de teste: loss = {:.6f} MAE = {:.2f} %".format(eval[0], eval[1]*100) )
```

```
Dados de treino: loss = 0.002600 MAE = 3.54 %
Dados de teste: loss = 0.001749 MAE = 2.70 %
```

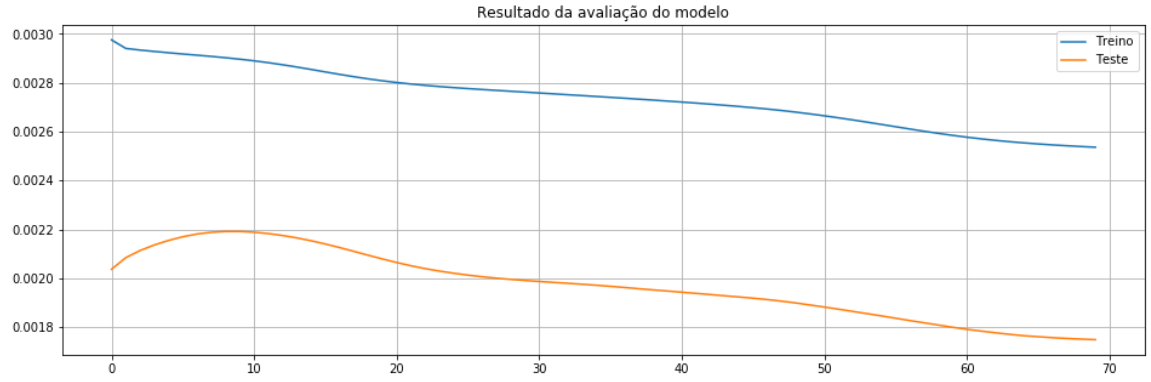
In [98]:

```
#CÉLULA TCC-86
# Salvando modelo em arquivo
print("Salvando o modelo em arquivo")
mp = "lstmTCC_model.h5"
model.save(mp)
```

```
Salvando o modelo em arquivo
```

In [99]:

```
#CÉLULA TCC-87
#Plotanto o history com dados de treino e teste
plt.figure(figsize=(16,5))
plt.plot(h.history['loss'], label='Treino')
plt.plot(h.history['val_loss'], label='Teste')
plt.legend()
plt.title("Resultado da avaliação do modelo")
plt.grid()
plt.show()
```



In [100]:

```
#CÉLULA TCC-88
#Realizando uma predição
y_pred_Train = model.predict(X_train)
y_pred_Test = model.predict(X_test)

X_train_r = X_train.reshape((X_train.shape[0], X_train.shape[2]))
X_test_r = X_test.reshape((X_test.shape[0], X_test.shape[2]))

# Invertendo a escala para previsão
inv_y_pred_Train = np.concatenate((y_pred_Train, X_train_r[:, 1:]), axis=1)#-6
#inv_y_pred_Train = min_max_scaler.inverse_transform(inv_y_pred_Train)
inv_y_pred_Train = inv_y_pred_Train[:,0]

inv_y_pred_Test = np.concatenate((y_pred_Test, X_test_r[:, 1:]), axis=1)#-6
#inv_y_pred_Test = min_max_scaler.inverse_transform(inv_y_pred_Test)
inv_y_pred_Test = inv_y_pred_Test[:,0]

# Invertendo a escala atual
y_train_r = y_train.reshape((len(y_train), 1))
inv_y_Train = np.concatenate((y_train_r, X_train_r[:, 1:]), axis=1)#-6
#inv_y_Train = min_max_scaler.inverse_transform(inv_y_Train)
inv_y_Train = inv_y_Train[:,0]

y_test_r = y_test.reshape((len(y_test), 1))
inv_y_Test = np.concatenate((y_test_r, X_test_r[:, 1:]), axis=1)#-6
#inv_y_Test = min_max_scaler.inverse_transform(inv_y_Test)
inv_y_Test = inv_y_Test[:,0]
```

In [101]:

```
#CÉLULA TCC-89
#Exibindo os resultados de treino e teste
print('-----------------------------------------------------')
print('------------          LSTM          ----------------')
print('-----------------------------------------------------')

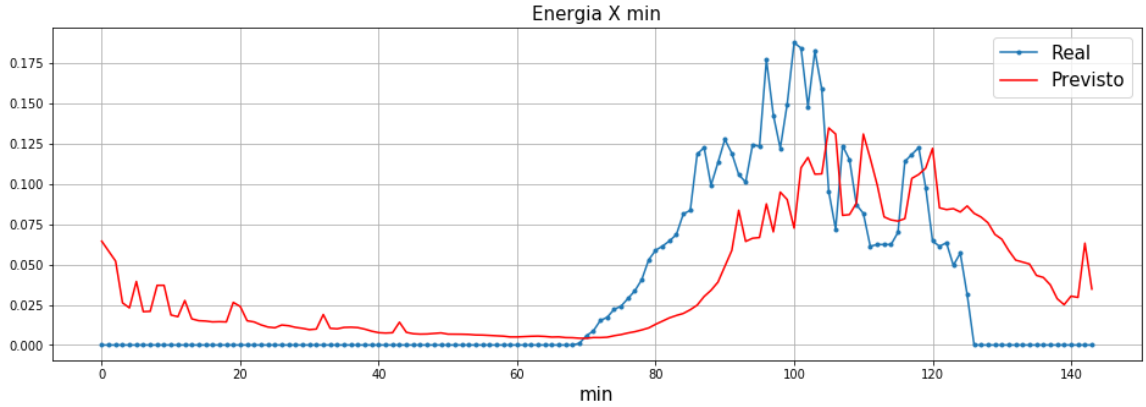
print('------------------- TREINO ------------------------')
print('MSE: {:.6f}'.format(metrics.mean_squared_error(inv_y_Train, inv_y_pred_Train)))
print('RMSE: {:.6f}'.format(np.sqrt(metrics.mean_squared_error(inv_y_Train, inv_y_pred_
Train))))
print('r²: {:.2f} %'.format(r2_score(inv_y_Train, inv_y_pred_Train) * 100))
print('MAE: {:.6f}'.format(mean_absolute_error(inv_y_Train, inv_y_pred_Train)))
print('------------------- TESTE -------------------------')
print('MSE: {:.6f}'.format(metrics.mean_squared_error(inv_y_Test, inv_y_pred_Test)))
print('RMSE: {:.6f}'.format(np.sqrt(metrics.mean_squared_error(inv_y_Test, inv_y_pred_T
est))))
print('r²: {:.2f} %'.format(r2_score(inv_y_Test, inv_y_pred_Test) * 100))
print('MAE: {:.6f}'.format(mean_absolute_error(inv_y_Test, inv_y_pred_Test)))
print('-----------------------------------------------------')
```

```
-----------------------------------------------------
------------          LSTM          ----------------
-----------------------------------------------------
------------------- TREINO ------------------------
MSE: 0.002600
RMSE: 0.050995
r²: 41.45 %
MAE: 0.035378
------------------- TESTE -------------------------
MSE: 0.001749
RMSE: 0.041819
r²: 40.46 %
MAE: 0.026966
-----------------------------------------------------
```

In [102]:

```
#CÉLULA TCC-90
#Exibindo um gráfico da previsão
#qtdRegPred = len(y_test)
qtdRegPred = 144

dataPlot=[x for x in range(qtdRegPred)]
plt.figure(figsize=(16,5))
plt.plot(dataPlot, y_test[:qtdRegPred], marker='.', label="Real")
plt.plot(dataPlot, y_pred_Test[:qtdRegPred], 'r', label="Previsto")
plt.xlabel('min', size=15)
plt.legend(fontsize=15)
plt.title('Energia X min', size=15)
plt.grid()
plt.show()
```



4 - Conclusão

Após as análises dos resultados com os quatro modelos utilizados, verificamos que com a remoção do atributo relacionado a radição solar, os valores das métricas de erro MSE, RMSE e MAE não apresentaram muita diferença aos modelos treinados com o campo em questão, o r² apresentou uma queda na qual acreditamos que significa que os modelos passam a explicar bem nenos dados em relação aos outros, porém, concluimos que os valores de erros apresentando valores baixos já é satisfatório para a previsão de enregia dos próximos dez minutos.

5 - Melhorias futuras

Acreditamos que com a obtenção de mais dados, completando assim mais de dois anos de dados, que o modelo de rede neural se comporte de uma maneira melhor, podendo prever corretamente os valores de geração de energia com uma maior facilidade e com um tempo a frente superior, não so dez min.

Referências

Foram utilizados os seguintes links como referências:

- https://towardsdatascience.com/linear-regression-in-python-9a1f5f000606 (https://towardsdatascience.com/linear-regression-in-python-9a1f5f000606)
- https://www.dataquest.io/blog/understanding-regression-error-metrics/ (https://www.dataquest.io/blog/understanding-regression-error-metrics/)
- https://machinelearningmastery.com/multivariate-time-series-forecasting-lstms-keras/ (https://machinelearningmastery.com/multivariate-time-series-forecasting-lstms-keras/)